"SYA: An Explainable and Responsible AI Platform Personalised Learning"

Minor Project Report Submitted
To
Chhattisgarh Swami Vivekanand Technical University, Bhilai

for
Completion of 7th Semester Minor Project of the degree
of
BACHELOR OF TECHNOLOGY (HONORS)
in
ARTIFICIAL INTELLIGENCE
(COMPUTER SCIENCE & ENGINEERING)

Submitted By

Durgesh Kumar Dewangan
Roll No. **300012723018** Enrollment No. **CC7457**
B.Tech. (Hons.) 7th Semester

Under the Guidance of
Mr. Siddharth Meshram
Assistant Professor
Department of Computer Science and Engineering
University Teaching Department, CSVTU, Bhilai (C.G.)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
UNIVERSITY TEACHING DEPARTMENT
CHHATTISGARH SWAMI VIVEKANAND TECHNICAL UNIVERSITY, BHILAI
Session: 2026 -2027

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
UNIVERSITY TEACHING DEPARTMENT
CHHATTISGARH SWAMI VIVEKANAND TECHNICAL UNIVERSITY, BHILAI

DECLARATION BY THE CANDIDATE

I, the undersigned, solemnly declare that the Minor Project entitled "**SYA: An Explainable and Responsible AI Platform for Personalised Learning**" represents my own independent academic effort and intellectual contribution, carried out during the VII semester of my graduation.

The project involved the design and implementation of a syllabus-driven learning platform incorporating retrieval-augmented generation, explainable artificial intelligence and responsible artificial intelligence validation. The software described in this report, including its backend services, web interface, evaluation layer and automated test suite, was designed, written and tested by me.

This work was undertaken under the supervision of **Mr. Siddharth Meshram** Assistant professor, Department of Computer Science and Engineering, **University Teaching Department, Chhattisgarh Swami Vivekanand Technical University, Bhilai (C.G.), India,** in alignment with the academic objectives of the Department.

I affirm that all ideas, analyses, results and interpretations presented herein are original and derived from my own understanding of the subject, except where explicitly cited and acknowledged. Where third-party open-source libraries have been used, they are identified in the report and listed in the references. The reported results are those actually produced by the implemented system; no result has been fabricated or overstated. This document has been prepared with integrity and in accordance with the academic standards expected by the institution.

(Signature of the Candidate)
Durgesh Kumar Dewangan
Roll No. 300012723018
Enrollment No. CC7457
Date: _____

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
UNIVERSITY TEACHING DEPARTMENT
CHHATTISGARH SWAMI VIVEKANAND TECHNICAL UNIVERSITY, BHILAI

CERTIFICATE BY THE SUPERVISOR

This is to certify that the Minor Project entitled "**SYA: An Explainable and Responsible AI Platform for Personalised Learning**" documents the project work carried out by **Durgesh Kumar Dewangan**, bearing Roll No. **300012723018** and Enrollment No. **CC7457**, under my guidance and supervision, in partial fulfilment of the requirements for the completion of the VII semester Minor Project of the degree of Bachelor of Technology (Honors) in Artificial Intelligence (Computer Science & Engineering) at Chhattisgarh Swami Vivekanand Technical University, Bhilai (C.G.), India.

To the best of my knowledge and belief, the project work:

- Represents the original effort and contribution of the candidate himself;
- Has been satisfactorily completed within the prescribed duration;
- Meets the academic and procedural requirements of the University's Ordinance for the B.Tech. (Honors) degree; and
- Is of an acceptable standard in both content and presentation for submission to the University and evaluation by the examiners.

(Signature of H.O.D.)

Dr. J.P. Patra
Associate Professor & HOD
Department of CSE

(Signature of the Supervisor)

Mr. Siddharth Meshram
Assistant Professor
UTD-CSVTU Bhilai

Forwarded to Chhattisgarh Swami Vivekanand Technical University, Bhilai (C.G.)

……
(Signature of the Director, UTD-CSVTU)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
UNIVERSITY TEACHING DEPARTMENT
CHHATTISGARH SWAMI VIVEKANAND TECHNICAL UNIVERSITY, BHILAI

CERTIFICATE BY THE EXAMINER

This is to certify that the Minor Project entitled "**SYA: An Explainable and Responsible AI Platform for Personalised Learning**" was submitted by **Durgesh Kumar Dewangan**, a student of B.Tech. (Honors) in Artificial Intelligence (Computer Science & Engineering), bearing Roll No. **300012723018** and Enrollment No. **CC7457**.

It has been examined by the undersigned as a part of the examination and is hereby recommended for the completion of the VII semester Minor Project for the degree of Bachelor of Technology (Honors) in Artificial Intelligence (Computer Science and Engineering) at Chhattisgarh Swami Vivekanand Technical University, Bhilai (C.G.), India.

…….	**……..**
Internal Examiner	**External Examiner**
Date:	Date:

ACKNOWLEDGEMENT

The real spirit of achieving a goal is through excellence and disciplined effort. I take this opportunity to express my gratitude to everyone who supported me through the course of this Minor Project.

With deep gratitude I acknowledge my indebtedness to my guide, **Mr. Siddharth Meshram** Assistant professor, Department of Computer Science and Engineering, University Teaching Department, CSVTU Bhilai, under whose guidance I had the privilege to complete this project work. His insistence that every claim in an engineering report be traceable to evidence shaped the direction of this project more than any other single factor: the trust-scoring and explainability layer that forms its core contribution exists because of that emphasis.

I am grateful to all the faculty members of the Department of Computer Science and Engineering, who were available in the hour of need and provided the help and facilities required for the completion of this work, and whose questions during the periodic project reviews materially improved the design of the retrieval and validation components.

I owe my sincere thanks to the Director, University Teaching Department, CSVTU Bhilai, for the inspiration and constant encouragement that enabled me to present my work in this form.

I also acknowledge the open-source community whose work this project builds upon, particularly the maintainers of FastAPI, SQLAlchemy, FAISS, sentence-transformers, NetworkX, PyMuPDF and pdfplumber, and the authors of the research on retrieval-augmented generation and model explanation cited in the references.

Durgesh Kumar Dewangan
Roll No. 300012723018
Enrollment No. CC7457

ABSTRACT

A syllabus is the single document common to every student of a programme, yet in its distributed form it is little more than an ordered list of topic names. It specifies what must be learned without supplying explanatory material, and without indicating the order in which the topics build upon one another. Learners increasingly fill this gap with general-purpose AI assistants, which produce fluent explanations drawn from model memory rather than from the learner's prescribed material. The resulting answers cannot be audited: a statement that is accurate and a statement that is fabricated are presented with identical confidence, which is unsafe in an assessment-driven academic setting.

This Minor Project presents SYA, a locally deployable web platform that converts an uploaded syllabus into a structured, prerequisite-ordered learning path and generates study material for each topic through a retrieval-augmented pipeline constrained to the learner's own documents. The system is organised as a five-phase pipeline: document processing, multi-strategy retrieval, language-model generation, explainable AI analysis and responsible AI validation. Document processing extracts text, tables and figures and indexes them as embeddings in a per-subject FAISS index. Retrieval combines dense vector search with a lexical BM25 implementation, fuses the two rankings by reciprocal rank fusion, re-ranks the candidates and selects context under a character budget. Generation constructs a numbered-evidence prompt and calls a configurable chain of language-model providers, falling back to an evidence-extracting template when none is available.

The distinguishing contribution is the evaluation layer. For every generated lesson the system computes retrieval attribution, per-chunk context relevance, exact Shapley values over the retrieved evidence, and a sentence-to-source evidence trace; it then measures groundedness, faithfulness, hallucination risk and semantic similarity, and combines them into a weighted trust score with a stated band. Metrics that cannot be computed are reported as unavailable with a stated reason rather than as a default value.

The platform was implemented in approximately 12,900 lines across 72 source files, exposing fifteen HTTP endpoints over a six-table relational schema, with a dependency-free JavaScript front end. Verification comprises 65 automated backend tests and a 13-check front-end harness, all passing. Measured on a controlled comparison against the same uploaded document, a lesson restating the source material scored a trust value of 0.889 (high band) while an off-topic lesson scored 0.052 (low band), with faithfulness of 1.000 against 0.000; a numeric value absent from the evidence was correctly flagged. The results demonstrate that generated study material can be bound to a learner's own documents and that the strength of that binding can be measured and displayed rather than assumed.

Keywords: Retrieval-Augmented Generation, Explainable AI, Responsible AI, Trust Score, Shapley Values, Hybrid Retrieval, Knowledge Graph, Educational Technology, FastAPI, FAISS.

TABLE OF CONTENTS

CHAPTER I INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Motivation 2
1.4 Aim of the Project 2
1.5 Objectives 2
1.5.1 Primary Objective 2
1.5.2 Specific Objectives 2
1.6 Scope of the Project 3
1.7 Intended Users 4
1.8 Key Contributions 4
1.9 Limitations 4
1.10 Organisation of the Report 5
CHAPTER II SYSTEM ANALYSIS 6
2.1 Existing Systems and Approaches 6
2.1.1 Manual Study Planning 6
2.1.2 Learning Management Systems 6
2.1.3 General-Purpose Conversational AI Assistants 6
2.1.4 Retrieval-Augmented Question Answering over Documents 6
2.2 Limitations of the Existing Approaches 6
2.3 Problems Identified 7
2.4 Proposed System 7
2.5 Functional Requirements 7
2.6 Non-Functional Requirements 8
2.7 Constraints 9
CHAPTER III FEASIBILITY STUDY AND REQUIREMENTS 10
3.1 Technical Feasibility 10
3.2 Operational Feasibility 10
3.3 Economic Feasibility 11
3.4 Schedule Feasibility 11
3.5 Hardware Requirements 11
3.6 Software Requirements 12
3.7 Assumptions 12
CHAPTER IV SYSTEM DESIGN AND ARCHITECTURE 13
4.1 Design Approach 13

4.2 System Architecture 13
4.3 Component Description 14
4.4 Data Flow Diagrams 14
4.4.1 DFD Level 0 - Context Diagram 14
4.4.2 DFD Level 1 15
4.4.3 DFD Level 2 15
4.5 Entity-Relationship Model 16
4.6 Use-Case Model 17
4.7 API Design 18
CHAPTER V METHODOLOGY AND ALGORITHMS 20
5.1 Methodological Overview 20
5.2 Phase 1 - Document Processing 20
5.2.1 Text Extraction 20
5.2.2 Curriculum Parsing 21
5.2.3 Multimodal Extraction 21
5.2.4 Chunking and Indexing 21
5.3 Phase 2 - Multi-Strategy Retrieval 22
5.3.1 Dense and Lexical Retrieval 22
5.3.2 Reciprocal Rank Fusion 22
5.3.3 Re-ranking and Context Selection 23
5.4 Phase 3 - Generation and Response Optimisation 23
5.4.1 Prompt Construction 23
5.4.2 Provider Chain and Diagnostics 23
5.4.3 Template Fallback 23
5.4.4 Response Optimisation and Strict Retry 24
5.5 Phase 4 - Explainable AI 24
5.5.1 Retrieval Attribution and Context Relevance 24
5.5.2 Shapley Values over Retrieved Evidence 24
5.5.3 Evidence Tracing 24
5.6 Phase 5 - Responsible AI Validation 25
5.6.1 Evidence Construction and the Similarity Backend 25
5.6.2 Validation Metrics 25
5.6.3 Composite Trust Score 25
5.7 Knowledge Graph Construction and Sequencing 26
CHAPTER VI IMPLEMENTATION AND MODULE DESCRIPTION 27
6.1 Implementation Overview 27
6.2 Module Interaction 27

6.3 Backend Module Specifications 28
6.3.1 Document Extraction 28
6.3.2 Curriculum Parser 28
6.3.3 Multimodal Extraction 28
6.3.4 Embeddings and Vector Store 29
6.3.5 Hybrid Retrieval 29
6.3.6 Similarity Backend 29
6.3.7 Language Model Client 30
6.3.8 Pipeline Orchestrator 30
6.3.9 Explainable AI 30
6.3.10 Responsible AI Validation 31
6.3.11 Knowledge Graph 31
6.3.12 API Routers 31
6.4 Front-End Implementation 32
6.5 Representative Implementation Excerpts 32
CHAPTER VII DATABASE AND DATA STRUCTURES 34
7.1 Database Technology 34
7.2 Schema Specification 34
7.3 Relationships and Integrity 36
7.4 Design Decisions and Trade-offs 36
7.5 Non-Relational Data Structures 36
CHAPTER VIII TESTING AND VALIDATION 38
8.1 Testing Strategy 38
8.2 Test Environment and Isolation 38
8.3 Test Case Specifications 39
8.4 Test Execution Results 40
8.5 Defects Identified and Corrected 41
8.6 Validation of the Evaluation Layer 42
CHAPTER IX RESULTS AND DISCUSSION 43
9.1 Implemented Functionality 43
9.2 System Operation 43
9.3 Evaluation of the Trust Scoring Layer 44
9.3.1 Interpretation 44
9.3.2 Limits of the Measurement 45
9.4 Behaviour Without Source Prose 45
9.5 Verification Results 45
9.6 Comparison Against the Stated Objectives 46

9.7 Known Issues and Limitations of the Result 46
CHAPTER X CONCLUSION AND FUTURE SCOPE 47
10.1 Conclusion 47
10.2 Objectives Achieved 47
10.3 Technical Contributions 47
10.4 Future Scope 48
10.4.1 Immediate Extensions 48
10.4.2 Scalability and Deployment 48
10.4.3 Functional Extensions 48
10.5 Concluding Remark 49
REFERENCE……..50
APPENDIX A: SCREENSHOT OF THE PROJECT……51
APPENDIX B: CONFIGURATION REFERENCE……...52

LIST OF FIGURES

Figure	Title of Figure
Fig 4.1	Overall system architecture
Fig 4.2	DFD Level 0 - context diagram
Fig 4.3	DFD Level 1 - major processes and data stores
Fig 4.4	DFD Level 2 - retrieval, generation, explanation and validation
Fig 4.5	Entity-relationship diagram
Fig 4.6	Use-case diagram
Fig 5.1	The five-phase processing pipeline
Fig 5.2	Hybrid retrieval and context selection
Fig 6.1	Module interaction map
Fig 8.1	Distribution of the 65 backend tests by test file
Fig 9.1	Measured trust metrics for a grounded against an off-topic lesson

LIST OF TABLES

Table	Title of Table
Table 1.1	Intended users and the needs addressed
Table 2.1	Comparison of existing approaches with the proposed system
Table 2.2	Functional requirements
Table 2.3	Non-functional requirements
Table 3.1	Technical feasibility by required capability
Table 3.2	Economic feasibility
Table 3.3	Work decomposition used for schedule feasibility
Table 3.4	Hardware requirements
Table 3.5	Software requirements
Table 4.1	System components and their responsibilities
Table 4.2	Use-case specifications
Table 4.3	HTTP API endpoints
Table 5.1	Similarity thresholds calibrated per method
Table 5.2	Trust score components and weights
Table 6.1	Source code distribution
Table 6.2	Module specification - Document Extraction
Table 6.3	Module specification - Curriculum Parser
Table 6.4	Module specification - Multimodal Extraction
Table 6.5	Module specification - Embeddings and Vector Store
Table 6.6	Module specification - Hybrid Retrieval
Table 6.7	Module specification - Similarity Backend
Table 6.8	Module specification - Language Model Client
Table 6.9	Module specification - Pipeline Orchestrator
Table 6.10	Module specification - Explainable AI
Table 6.11	Module specification - Responsible AI Validation
Table 6.12	Module specification - Knowledge Graph
Table 6.13	Module specification - API Routers
Table 6.14	Front-end modules
Table 7.1	users table
Table 7.2	syllabi table
Table 7.3	units table
Table 7.4	topics table
Table 7.5	subtopics table
Table 7.6	reference_sources table
Table 7.7	Entity relationships and integrity rules
Table 7.8	Non-relational data structures
Table 8.1	Levels of testing applied
Table 8.2	Test case specifications and outcomes
Table 8.3	Backend test distribution and outcome
Table 8.4	Defects identified during testing and their correction
Table 9.1	Implemented functionality and its verification status
Table 9.2	Trust metrics for a grounded and an off-topic lesson on identical evidence
Table 9.3	Achievement of the stated objectives
Table 10.1	Configuration settings

LIST OF ABBREVIATIONS

S. No.	Abbreviation	Full Form
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	BM25	Best Matching 25 (lexical ranking function)
4	BLIP	Bootstrapping Language-Image Pre-training
5	CORS	Cross-Origin Resource Sharing
6	CRUD	Create, Read, Update, Delete
7	CSV	Comma-Separated Values
8	DAG	Directed Acyclic Graph
9	DFD	Data Flow Diagram
10	ER	Entity-Relationship
11	FAISS	Facebook AI Similarity Search
12	HTTP	Hypertext Transfer Protocol
13	IDF	Inverse Document Frequency
14	JSON	JavaScript Object Notation
15	LIME	Local Interpretable Model-Agnostic Explanations
16	LLM	Large Language Model
17	NLP	Natural Language Processing
18	OCR	Optical Character Recognition
19	ORM	Object-Relational Mapping
20	PDF	Portable Document Format
21	rAI	Responsible Artificial Intelligence
22	RAG	Retrieval-Augmented Generation
23	REST	Representational State Transfer
24	RRF	Reciprocal Rank Fusion
25	SHAP	SHapley Additive exPlanations
26	SQL	Structured Query Language
27	TF-IDF	Term Frequency-Inverse Document Frequency
28	UI	User Interface
29	UUID	Universally Unique Identifier
30	xAI	Explainable Artificial Intelligence

CHAPTER I
INTRODUCTION

1.1 Background

Every student enrolled in a university programme receives a syllabus. It is the one authoritative description of what a course covers, and it is usually the first document a learner consults when planning study. In the form in which it is distributed, however, a syllabus is close to the least useful document a learner owns. It is typically a sequence of unit headings followed by bare topic names: "Unit 2: Data Structures - Arrays, Linked Lists, Hash Tables". It states what must be learned. It does not explain any of it, it does not indicate how long any topic should take, and it does not say which topics must be understood before others can be attempted.

The learner must therefore convert the syllabus into a study plan manually, and then find material for each topic. Traditionally this meant textbooks and lecture notes. Increasingly it means a general-purpose conversational AI assistant, because such assistants are immediate, free at the point of use, and will produce an explanation of any topic on request.

That substitution introduces a problem that the technology itself does not solve. A large language model generates text from parameters learned during training; it is not, by default, consulting the learner's prescribed material. The explanation it produces may be correct, may be correct but out of scope for the syllabus being studied, or may be confidently wrong. All three outcomes are rendered in the same fluent, authoritative prose. The learner has no mechanism to distinguish them, and in an assessment-driven setting the cost of the third case is borne entirely by the learner.

Retrieval-augmented generation (RAG), introduced by Lewis et al. [3], addresses part of this by retrieving relevant passages from a document collection and supplying them to the model as context. Grounding generation in retrieved evidence demonstrably reduces unsupported output. It does not, by itself, tell the reader how well grounded any particular sentence turned out to be. A RAG system that retrieves poor evidence, or that generates text only loosely related to the evidence it retrieved, still produces confident prose. The gap this project addresses is not the absence of retrieval; it is the absence of a visible, quantitative account of how far the output can be trusted.

1.2 Problem Statement

Learners possess a syllabus that specifies their curriculum but provides no learning material, no ordering and no explanation. Available AI study tools generate material readily but offer no verifiable connection between what they generate and what the learner is actually required to study, and no measure of the reliability of any individual statement. There is consequently no way for a learner, or for an instructor reviewing the learner's materials, to audit AI-generated study content.

The problem addressed by this project is therefore stated as follows: to design and implement a learning platform that (i) converts an unstructured syllabus document into a structured, prerequisite-ordered learning path without manual data entry, (ii) generates study material for each topic that is constrained to and traceable back to the learner's own uploaded documents, and (iii) computes and displays an auditable measure of how strongly each generated lesson is supported by that material, including the identification of individual statements which are not supported.

1.3 Motivation

Three observations motivated the work. First, the syllabus is an underused asset: it is already in the learner's possession, already authoritative, and already machine-readable in most cases, yet no common tool converts it into anything actionable. Second, the failure mode of generative study tools is silent. A tool that is wrong and says so is manageable; a tool that is wrong and indistinguishable from being right is not. Third, the techniques required to close that gap - dense and lexical retrieval, rank fusion, attribution methods such as Shapley values [2], and similarity-based groundedness checking - are individually well established in the literature but are rarely assembled into a single system whose output a non-specialist can inspect.

The project therefore treats measurement of trustworthiness as a first-class feature rather than an auxiliary diagnostic. The design principle followed throughout is that the system must never present a number it cannot justify: where a metric cannot be computed, the system reports that it is unavailable, and why, instead of substituting a default.

1.4 Aim of the Project

To design, implement and validate a locally deployable web platform that transforms an uploaded syllabus into an ordered, topic-wise learning path, generates study material for each topic through a retrieval-augmented pipeline restricted to the learner's own material, and accompanies every generated lesson with a quantitative trust score and a complete, inspectable explanation of the evidence behind it.

1.5 Objectives

1.5.1 Primary Objective

To build a working syllabus-to-lesson pipeline in which every generated lesson carries a computed trust score derived from a comparison of the generated text against the evidence retrieved from the learner's own documents.

1.5.2 Specific Objectives

1. **Document processing.** To extract text, tables and figure descriptions from uploaded PDF, plain-text and image documents, using a staged fallback so that a failure of one extraction method does not abort the upload.
2. **Curriculum parsing.** To derive a subject, unit and topic hierarchy from unstructured syllabus text, handling both single-subject and multi-subject documents, and to assign each topic a stable, globally unique identifier.
3. **Prerequisite sequencing.** To construct a directed knowledge graph over the parsed curriculum and derive, by topological sort, a learning order in which each topic carries its step number and its immediate prerequisite.
4. **Hybrid retrieval.** To implement dense vector retrieval over a per-subject FAISS index together with a lexical BM25 retriever, to fuse the two rankings using reciprocal rank fusion, and to select a context set under an explicit character budget with near-duplicate suppression.
5. **Grounded generation.** To construct prompts that present retrieved evidence as numbered references, to call a configurable chain of language-model providers, and to fall back to an evidence-extracting template when no provider is available.
6. **Explainable AI analysis.** To compute retrieval attribution per source document,

per-chunk context relevance, exact Shapley values quantifying each evidence chunk's contribution, and a sentence-level trace linking generated text to its supporting chunk.

7. **Responsible AI validation.** To compute groundedness, faithfulness, hallucination risk and semantic similarity, to classify every generated sentence as supported, weak or unsupported, and to combine the metrics into a single weighted trust score with a declared band.
8. **Graceful degradation.** To ensure the system remains functional when the embedding model, OCR engine or language-model provider is unavailable, and to report the degraded mode in the output rather than concealing it.
9. **Verification.** To validate the implementation with an automated test suite covering parsing, graph sequencing, retrieval behaviour, the explainability mathematics, validation scoring, provider handling and the HTTP interface.

1.6 Scope of the Project

The project scope, as implemented, covers the following.

- Ingestion of syllabus documents in PDF, plain-text, PNG and JPEG formats, with optional OCR for scanned documents.
- Automatic extraction of a subject-unit-topic curriculum, including multi-subject documents.
- Construction of a prerequisite knowledge graph and derivation of a learning roadmap.
- Per-subject vector indexing of the uploaded material, plus up to two additional reference documents per subject.
- Hybrid retrieval, prompt construction and lesson generation, with four supported language-model providers and a template fallback.
- Generation of practice activities (multiple-choice, definition arrangement, fill-in-the-blank and term classification) alongside each lesson.
- Computation and display of the full explainability and validation report for every lesson.
- Recording of topic completion and accuracy, persisted server-side.
- A browser-based interface requiring no build step and no account system.

The following are explicitly outside the scope of this project.

- Multi-user authentication, authorisation and role management. The data model reserves a user table, but the application is single-user and local.
- Training or fine-tuning of language models. The system consumes models through an HTTP interface and does not modify them.
- Mobile native applications. The interface is responsive but browser-based.
- Institutional deployment behind a load balancer. The per-topic generation lock is in-process and correct for a single server process only.
- Verification of factual correctness against the world. The system measures support relative to the learner's uploaded material, which is a different and narrower claim.

1.7 Intended Users

Table 1.1: Intended users and the needs addressed

User	Need addressed by the system
Undergraduate student	Converts a distributed syllabus into an ordered study path with explanatory material for each topic, and shows which parts of that material are backed by the prescribed documents.
Student preparing for assessment	Identifies statements that are not supported by the uploaded material, so that revision effort can be directed at content requiring verification.
Course instructor	Allows inspection of the evidence and trust score behind AI-generated study material before recommending such tools, and highlights syllabus areas where the available material is too thin to support reliable generation.
Self-learner	Provides structure for an independent course of study built from the learner's own collected documents.

1.8 Key Contributions

1. **An end-to-end syllabus-to-lesson pipeline.** A complete implementation from raw document upload to rendered, activity-bearing lesson, with no manual curriculum entry at any stage.
2. **A native exact Shapley-value implementation for evidence attribution.** Shapley values are computed over the retrieved evidence set, with the characteristic function defined as the mean best-match similarity of the generated sentences to the chunks in a coalition. The implementation is exact for evidence sets within a configured bound and switches to permutation sampling beyond it; its correctness is verified against the brute-force definition and the efficiency, symmetry and dummy-player axioms.
3. **A composite trust score with declared weights and honest unavailability.** Five normalised components are combined under published weights into a single score and band. Where no evidence or no generated claim exists, the score is reported as unavailable with a reason, never as zero.
4. **Method-aware similarity with calibrated thresholds.** The similarity backend selects sentence embeddings when available and TF-IDF cosine similarity otherwise, applies thresholds calibrated separately for each method, and reports in the output which method was actually used.
5. **Verified graceful degradation.** Each optional dependency has a defined fallback that is exercised by the test suite, and the degraded mode is surfaced in the user-visible report.

1.9 Limitations

The limitations below are stated at the outset because several of them constrain how the results in Chapter 9 should be read.

- **Support is not truth.** An "unsupported" verdict means that a statement is not corroborated by the retrieved material. A lesson may legitimately include correct general knowledge that the syllabus does not contain. The metric flags material for verification; it does not prove falsity.
- **Grounding requires substantive source material.** A syllabus consisting only of topic names contains no prose to ground against. In that case the system correctly

reports that no trust score can be computed rather than producing one.

- **Similarity-based validation, not entailment.** Groundedness and faithfulness are computed from semantic similarity, not from a trained natural-language-inference model. Paraphrase is handled well; subtle logical negation is not guaranteed to be detected.
- **Explanation scope.** The Shapley analysis explains the contribution of retrieved evidence to the lesson's groundedness. It does not explain the internal computation of the language model, which is accessed over HTTP as a black box.
- **Single-process concurrency control.** Duplicate-generation suppression uses an in-process lock, which is correct for the single-process deployment targeted but not for a multi-process deployment.
- **Heuristic components.** Image-type classification uses pixel statistics rather than a trained classifier, and topic-duration estimation uses a keyword heuristic. Both are identified as heuristics wherever their output is displayed.

1.10 Organisation of the Report

The remainder of this report is organised as follows.

- **Chapter II - System Analysis.** Reviews existing approaches, establishes their limitations, derives the proposed system and states the functional and non-functional requirements.
- **Chapter III - Feasibility Study and Requirements.** Assesses technical, operational, economic and schedule feasibility and specifies the hardware and software environment.
- **Chapter IV - System Design and Architecture.** Presents the layered architecture, data flow diagrams at three levels, the entity-relationship model, the use-case model and the API design.
- **Chapter V - Methodology and Algorithms.** Describes the five-phase pipeline and specifies each significant algorithm with its inputs, processing, outputs and rationale.
- **Chapter VI - Implementation and Module Description.** Documents each implemented module with its responsibility, interfaces, dependencies, error handling and status.
- **Chapter VII - Database and Data Structures.** Specifies the relational schema and the in-memory and on-disk data structures.
- **Chapter VIII - Testing and Validation.** Presents the testing strategy, the test-case inventory and the defects found and fixed.
- **Chapter IX - Results and Discussion.** Reports measured outcomes, compares them against the stated objectives and discusses known issues.
- **Chapter X - Conclusion and Future Scope.** Summarises the work and identifies extensions.

CHAPTER II
SYSTEM ANALYSIS

2.1 Existing Systems and Approaches

Four broad approaches are currently available to a learner who wishes to study from a syllabus. Each is considered below in terms of what it provides and where it falls short for the problem stated in Section 1.2.

2.1.1 Manual Study Planning

The learner reads the syllabus, locates textbook sections or lecture notes for each topic, and devises an order of study. The material consulted is authoritative and the learner retains full control. The approach is, however, entirely manual; it scales poorly across several subjects, it depends on the learner correctly inferring prerequisite relationships that the syllabus does not state, and it produces no reusable artefact.

2.1.2 Learning Management Systems

Institutional learning management systems host course materials against a course structure. They are reliable and integrated with assessment, but the structure and the content are both authored manually by staff. An LMS does not generate explanatory material and cannot convert an arbitrary uploaded syllabus into a learning path.

2.1.3 General-Purpose Conversational AI Assistants

A learner pastes a topic name into a general-purpose assistant and receives an explanation. This is immediate and requires no setup, which accounts for its adoption. The limitations are those identified in Section 1.1: the response is generated from model parameters rather than from the learner's prescribed material, there is no citation to any source the learner can check, there is no measure of reliability, and no persistent structure is produced. The learner also has no assurance that the explanation matches the depth or scope their syllabus demands.

2.1.4 Retrieval-Augmented Question Answering over Documents

Tools that index a user's documents and answer questions against them represent the closest existing approach. Retrieval grounds the answer in material the user supplied, and many such tools cite the passages retrieved. This addresses the provenance problem in part. Two gaps remain relevant to this project. First, these tools are question-answering interfaces: they respond to queries but do not derive a curriculum or an ordered learning path from the document. Second, citing the retrieved passage establishes what was retrieved, not how strongly the generated answer is actually supported by it; a system can retrieve an appropriate passage and still generate statements that passage does not support.

2.2 Limitations of the Existing Approaches

Table 2.1: Comparison of existing approaches with the proposed system

Approach	Structure from syllabus	Grounded in learner's material	Per-statement verification	Effort
Manual study planning	Manual	Yes	Manual	High
Learning management system	Manual, by staff	Yes	Not applicable	High (staff)
General-purpose AI assistant	No	No	No	Low

Approach	Structure from syllabus	Grounded in learner's material	Per-statement verification	Effort
RAG question answering	No	Yes	Partial (citation only)	Low
Proposed system (SYA)	Automatic	Yes	Yes, per sentence with a score	Low

The comparison in Table 2.1 is made on the four capabilities that follow directly from the problem statement. It is not a claim that the proposed system is superior to the alternatives in all respects: a learning management system offers institutional integration and assessment that this project does not attempt, and a general-purpose assistant has a breadth of knowledge that no locally retrieved corpus can match.

2.3 Problems Identified

1. The syllabus, although universally available and machine-readable, is not converted into an actionable structure by any commonly used tool.
2. Prerequisite relationships between topics are left implicit, so learners may attempt topics out of a sensible order.
3. Generated study material is not constrained to the learner's prescribed material, so its scope may not match the course.
4. Where retrieval is used, the connection between the retrieved evidence and each generated statement is not quantified.
5. No reliability signal accompanies generated content, so the learner cannot triage which statements require verification.
6. Failures of optional components, such as an unavailable embedding model, are typically silent, so output quality can degrade without the user being informed.

2.4 Proposed System

The proposed system accepts a syllabus document and processes it through five phases. Phase 1 extracts the text, tables and figures and indexes them. Phase 2 retrieves evidence for a given topic using both dense and lexical retrieval. Phase 3 constructs a prompt in which the retrieved evidence is presented as numbered references and obtains a lesson from a language-model provider, or from an evidence-extracting template if none is configured. Phase 4 analyses the result to determine which evidence contributed and to what degree. Phase 5 validates the result against the evidence and produces the trust score. The lesson, the score and the full evidence report are then presented together.

Two design commitments distinguish the proposal from a conventional RAG implementation. The first is that the evaluation output is part of the product rather than a developer diagnostic: the learner sees the trust score on the lesson itself and can open the full report. The second is that the system is required to be honest about its own degradation; every optional dependency has a defined fallback, and the fallback that was actually taken is reported in the output.

2.5 Functional Requirements

Table 2.2: Functional requirements

ID	Requirement	Priority
FR-01	The system shall accept uploads in PDF, plain-text, PNG and JPEG formats and reject other formats with a readable message.	High

ID	Requirement	Priority
FR-02	The system shall extract text from an uploaded document, attempting a sequence of extraction strategies before reporting failure.	High
FR-03	The system shall parse the extracted text into subjects, units and topics, supporting documents containing more than one subject.	High
FR-04	The system shall assign each topic a globally unique identifier and resolve collisions between separate uploads.	High
FR-05	The system shall construct a knowledge graph and derive a topological learning order, recording each topic's sequence position and immediate prerequisite.	High
FR-06	The system shall report upload progress through the extract, distribute and generate stages.	Medium
FR-07	The system shall index the extracted text as embeddings in a per-subject vector index, and shall retain chunk text so that lexical retrieval remains possible without embeddings.	High
FR-08	The system shall allow up to two additional reference documents per subject to be uploaded and indexed.	Medium
FR-09	The system shall retrieve evidence for a topic using both vector and keyword retrieval and shall fuse the rankings.	High
FR-10	The system shall generate a lesson comprising multiple subtopic sections, each with a stated source.	High
FR-11	The system shall generate practice activities for each topic.	Medium
FR-12	The system shall cache generated lessons and regenerate only on explicit request.	High
FR-13	The system shall compute source attribution, context relevance, Shapley values and a sentence-level evidence trace for each lesson.	High
FR-14	The system shall compute groundedness, faithfulness, hallucination risk, semantic similarity and a composite trust score with a band.	High
FR-15	The system shall classify each generated sentence as supported, weak or unsupported and shall list those requiring verification.	High
FR-16	The system shall report a metric as unavailable, with a reason, where it cannot be computed.	High
FR-17	The system shall record topic completion and accuracy persistently.	Medium
FR-18	The system shall expose the knowledge graph and the learning roadmap through the API.	Medium

2.6 Non-Functional Requirements

Table 2.3: Non-functional requirements

ID	Category	Requirement
NFR-01	Usability	The learner shall be able to move from upload to a rendered lesson without configuration beyond starting the two processes.
NFR-02	Transparency	Every displayed metric shall be traceable to a stated computation, and the similarity method used shall be reported.
NFR-03	Reliability	Failure of any optional dependency shall degrade functionality rather than abort the request, and the degradation shall be reported.
NFR-04	Performance	A cached lesson shall be served without re-invoking the generation pipeline.
NFR-05	Portability	The system shall run on any platform providing Python 3.10 or later and a modern browser, without a build step for the front end.
NFR-06	Maintainability	Each pipeline phase shall be implemented in a separate module with a defined interface, and shall be independently testable.
NFR-07	Data integrity	Concurrent requests for the same ungenerated topic shall produce exactly one generation call.
NFR-08	Privacy	Uploaded documents and generated content shall remain on the local machine, except where the learner explicitly configures a cloud language-model provider.
NFR-09	Testability	The automated test suite shall run offline and deterministically, without dependence on network availability.
NFR-10	Correctness of evaluation	The explainability mathematics shall be verified against its formal definition.

2.7 Constraints

- **Technical.** Dense retrieval requires an embedding model, which must be downloaded once; where it cannot be obtained, the system must operate on lexical retrieval alone.
- **Operational.** The target deployment is a single machine running one server process, which bounds the concurrency design.
- **Economic.** No budget is available for paid language-model usage, so the system must be fully functional with a local provider or with no provider at all.
- **Academic.** The work is bounded by a single semester and by the reporting requirements of the university guidelines.

CHAPTER III
FEASIBILITY STUDY AND REQUIREMENTS

3.1 Technical Feasibility

Technical feasibility was assessed by identifying, for each capability the system requires, a specific mature implementation available under a permissive licence. The assessment is reproduced in Table 3.1, and has since been confirmed by the completed implementation.

Table 3.1: Technical feasibility by required capability

Capability required	Selected technology	Feasibility assessment
HTTP API with validation and documentation	FastAPI with Pydantic	Feasible. Provides request validation and automatically generated interactive documentation, removing the need for a separate API documentation effort.
Relational persistence without a server	SQLAlchemy over SQLite	Feasible. A file-based database removes installation of a database server, which matters for a locally deployed single-user tool.
Dense vector similarity search	FAISS with sentence-transformers	Feasible. An inner-product index over normalised 384-dimensional vectors gives cosine similarity; the all-MiniLM-L6-v2 model is approximately 80 MB and runs on CPU.
Lexical retrieval	BM25, implemented natively	Feasible. The ranking function is short to implement, has no dependency, and provides the fallback path when embeddings are unavailable.
Document text, table and figure extraction	PyMuPDF, pdfplumber, Pillow	Feasible. PyMuPDF is fast for text layers, pdfplumber handles table structure, and optional PaddleOCR covers scanned documents.
Prerequisite ordering	NetworkX	Feasible. Provides directed graph construction, topological sorting and cycle detection.
Attribution and validation mathematics	NumPy, implemented natively	Feasible. Shapley values and the similarity metrics are implemented directly, which also makes them directly testable against their definitions.
Lesson generation	Ollama, Mistral, Groq or Gemini over HTTP	Feasible, with the qualification that the system must remain useful when none is configured; this drove the template fallback requirement.
User interface	HTML5, CSS3 and JavaScript ES modules	Feasible. Avoiding a framework removes the build toolchain entirely, so the front end is served as static files.

3.2 Operational Feasibility

The system is operationally feasible for its intended user because it imposes no administrative burden. There is no account to create, no server to provision and no external service to register with for the core workflow. Installation is performed by a single script that creates a virtual environment, installs dependencies, pre-caches the embedding model and verifies each component, reporting the status of each. Operation consists of starting two local processes and opening a browser.

The interaction model follows the structure the learner already has in mind - subject, then unit, then topic - so no new conceptual model must be learned. The explainability output is presented progressively: a single trust percentage is shown on the lesson itself, and the full report is available behind one interaction, so a learner who does not want the detail is not obstructed by it.

3.3 Economic Feasibility

Table 3.2: Economic feasibility

Cost head	Amount	Remark
Development hardware	Nil (existing)	Developed on a personal computer already owned.
Software licences	Nil	All libraries are open-source under permissive licences.
Database licence	Nil	SQLite is public domain.
Embedding model	Nil	all-MiniLM-L6-v2 is openly available; approximately 80 MB, downloaded once.
Language-model usage	Nil to optional	Local inference through Ollama is free; cloud providers offer free tiers. With no provider configured the system still operates.
Hosting	Nil	Runs locally; no hosting is required.
Total direct cost	Nil	The project cost is the developer's time.

The economic case therefore rests on the absence of recurring cost rather than on a return-on-investment calculation. This was a deliberate design constraint: a study tool that a student cannot afford to run continuously is not a usable study tool.

3.4 Schedule Feasibility

The work was decomposed along the five pipeline phases, each of which has a defined input and output and can therefore be implemented and tested before the next is begun. This sequencing made the schedule feasible within one semester, because an incomplete later phase never blocked verification of an earlier one.

Table 3.3: Work decomposition used for schedule feasibility

Phase	Work item	Outcome
1	Document processing, parsing and indexing	Upload to structured curriculum, with the text indexed for retrieval.
2	Hybrid retrieval	Evidence selection for a topic, verifiable independently of generation.
3	Generation and provider handling	Lesson production, with the template fallback ensuring an output in all cases.
4	Explainable AI analysis	Attribution, relevance, Shapley values and the evidence trace.
5	Responsible AI validation and interface	Trust scoring, the report renderer and the learning interface.
6	Verification and hardening	Automated test suite, defect correction and installation tooling.

3.5 Hardware Requirements

Table 3.4: Hardware requirements

Component	Minimum	Development and testing configuration
Processor	Dual-core x86-64, 2.0 GHz	Multi-core x86-64
Main memory	4 GB	8 GB or higher
Secondary storage	2 GB free	5 GB free: embedding model approximately 80 MB, plus indexes, uploads and the database
Graphics processor	Not required	Not required. CPU-only PyTorch is installed deliberately, since a CUDA build adds several gigabytes and provides no benefit at this scale

Component	Minimum	Development and testing configuration
Network	Not required for core operation	Required once to download the embedding model, and thereafter only for optional cloud providers or topic images
Display	1366 x 768	1920 x 1080

3.6 Software Requirements

Table 3.5: Software requirements

Layer	Software and version	Role in the system
Operating system	Windows 11 (development); any OS with Python 3.10+	Host platform
Language runtime	Python 3.13.7	Backend implementation language
Web framework	FastAPI 0.115.0	HTTP routing, request validation, API documentation
Application server	uvicorn 0.30.6	ASGI server
ORM	SQLAlchemy 2.0.32	Declarative models and session management
Database	SQLite (file-based)	Relational persistence
Vector index	faiss-cpu 1.9.0.post1	Dense similarity search per subject
Embeddings	sentence-transformers 3.0.1, all-MiniLM-L6-v2	384-dimensional sentence embeddings
Numerical computing	NumPy	Similarity matrices and Shapley computation
PDF processing	PyMuPDF, pdfplumber 0.11.4	Text, image and table extraction
Image processing	Pillow	Image statistics for figure classification
NLP	spaCy 3.8.7	Sentence segmentation during parsing
Graph	NetworkX 3.3	Knowledge graph and topological ordering
File upload	python-multipart, aiofiles	Multipart parsing and asynchronous file I/O
Configuration	pydantic-settings, python-dotenv	Typed settings from environment
Front end	HTML5, CSS3, JavaScript ES modules	Interface; no framework, no build step
Backend testing	pytest, httpx, reportlab	Test runner, test client, PDF fixtures
Front-end testing	Node.js	Dependency-free smoke harness
Optional OCR	paddlepaddle, paddleocr	Text extraction from scanned documents
Optional LLM	Ollama, Mistral, Groq or Gemini	Lesson generation

3.7 Assumptions

- The uploaded syllabus contains a recognisable unit or chapter structure, or can be treated as a single unit.
- A PDF upload either contains a text layer or OCR support has been installed.
- The learner's uploaded material is in English; sentence segmentation and the embedding model are configured for English.
- The machine has sufficient free storage for the embedding model and the generated indexes.
- For meaningful trust scores, the uploaded material contains explanatory prose and not only topic headings.

CHAPTER IV
SYSTEM DESIGN AND ARCHITECTURE

4.1 Design Approach

The system is designed as four horizontal layers with a strictly downward dependency rule: the presentation layer depends on the API layer, the API layer on the services, and the services on persistence. No layer calls upward. The consequence that matters in practice is that the five pipeline phases are pure service-layer code with no knowledge of HTTP, which is what allows the mathematics in Phases 4 and 5 to be tested directly against its definition without constructing a web request.

A second design decision concerns where the pipeline is orchestrated. Rather than distributing the sequence across the API handlers, a single function, run_topic_pipeline(), executes Phases 2 to 5 for one topic and assembles the report. Every caller - the on-demand endpoint, the pre-generation step that runs after upload, and the background task that prepares the next topic - uses that one function, so the behaviour cannot diverge between entry points.

4.2 System Architecture

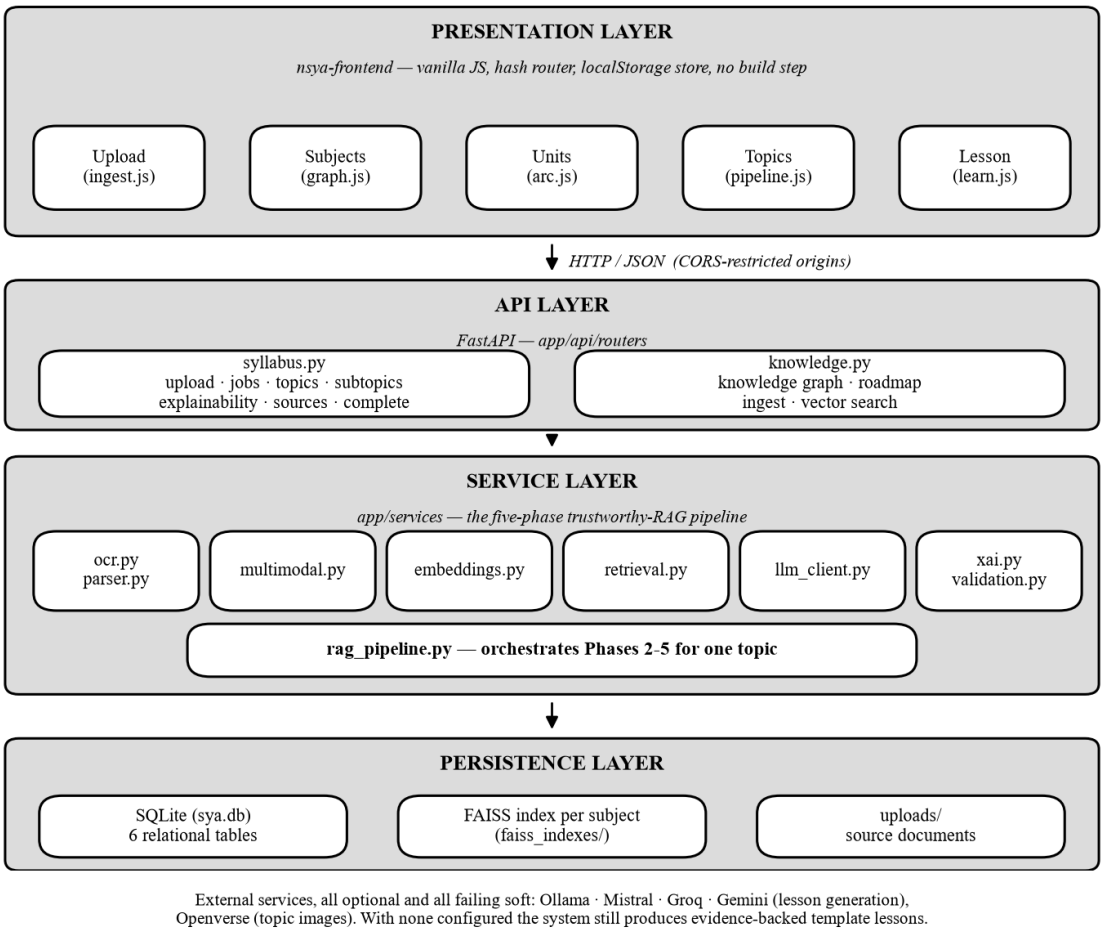


Fig 4.1: Overall system architecture

Figure 4.1 shows the layered architecture. The presentation layer is a set of ES modules served as static files; it holds no business logic beyond view state and talks to the backend

exclusively over JSON. The API layer exposes fifteen endpoints across two routers. The service layer contains the thirteen modules that implement the pipeline. The persistence layer comprises the relational database, the per-subject vector indexes and the uploaded files.

External services appear at the edge of the diagram and are all optional. Each has a defined fallback: absence of a language-model provider selects the evidence-extracting template, absence of the embedding model selects keyword-only retrieval, and absence of the image service simply omits the illustration. No external service is on the critical path for producing a lesson.

4.3 Component Description

Table 4.1: System components and their responsibilities

Layer	Component	Responsibility
Presentation	Screen modules	Five routed screens - upload, subjects, units, topics and lesson - plus a profile view.
Presentation	Store and router	A publish-subscribe state container persisted to browser storage, and a hash router that requires no server rewriting.
Presentation	Report renderer	A single shared renderer for the Phase 4 and 5 report, used by both the per-lesson popup and the per-unit drawer so the two cannot diverge.
API	Syllabus router	Upload, job status, topic and subtopic retrieval, explainability, reference sources and completion recording.
API	Knowledge router	Knowledge graph, learning roadmap, manual ingestion and manual vector search.
Service	Phase 1 modules	Text extraction, curriculum parsing, multimodal extraction, chunking and indexing.
Service	Phase 2 module	Hybrid retrieval and context selection.
Service	Phase 3 modules	Prompt construction, provider chain, response optimisation.
Service	Phase 4 module	Attribution, relevance, Shapley values, evidence tracing.
Service	Phase 5 module	Groundedness, faithfulness, hallucination risk, trust score.
Service	Support modules	Similarity backend with automatic fallback; knowledge graph construction; optional image lookup.
Persistence	Relational database	Six tables holding curriculum, content and progress.
Persistence	Vector indexes	One FAISS index and metadata store per subject.
Persistence	Upload store	Original documents, served read-only for source linking.

4.4 Data Flow Diagrams

4.4.1 DFD Level 0 - Context Diagram

At the context level the entire platform is a single process. The learner supplies a syllabus document and receives a learning path, lessons and trust reports. The only other external entity is the language-model provider, which receives a grounded prompt and returns a lesson in JSON. The diagram makes explicit that no other external system is involved in the core function, which is the basis of the privacy property stated in NFR-08.



Fig 4.2: DFD Level 0 - context diagram

4.4.2 DFD Level 1

Level 1 decomposes the platform into six processes over three data stores. Processes 1.0 to 3.0 run once per upload: document processing extracts the text, curriculum parsing derives the subject-unit-topic structure, and knowledge graph construction computes the sequence and prerequisite for every topic. Processes 4.0 and 5.0 run per topic, on first access, and are where retrieval, generation, explanation and validation occur. Process 6.0 records completion.

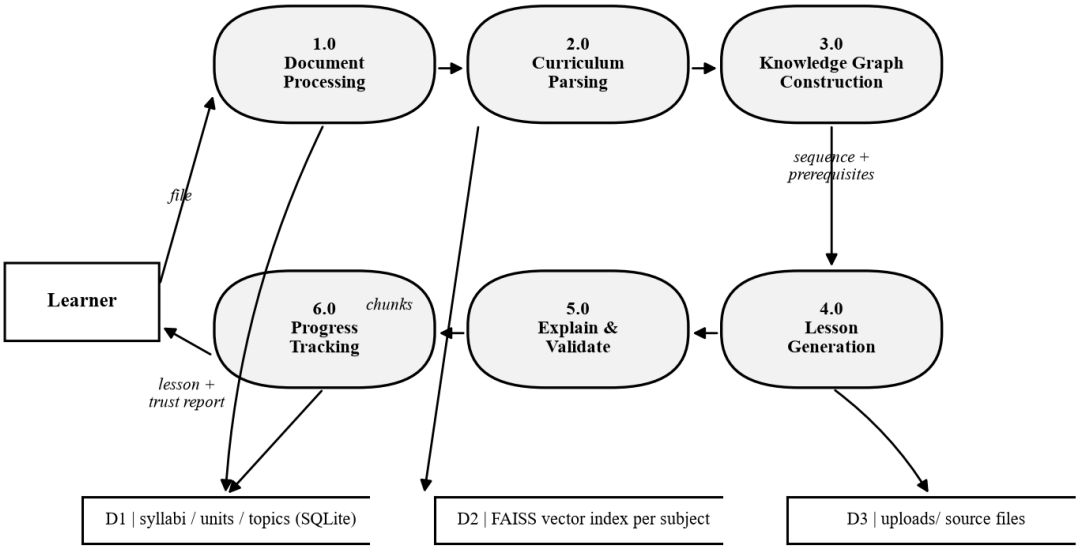


Fig 4.3: DFD Level 1 - major processes and data stores

4.4.3 DFD Level 2

Level 2 explodes process 4.0, which carries most of the system's logic. The query is formed from the topic and unit titles. Two retrievers run against the subject's index: dense vector search and BM25. Their rankings are fused, re-ranked and reduced to a context set. The

prompt is constructed with that context presented as numbered evidence and sent to the provider chain. The response is normalised, then analysed by the explainability stage and scored by the validation stage. If the resulting trust score falls below the configured threshold and the lesson came from a real provider, one further generation is attempted under a stricter prompt, and the better of the two drafts is kept.

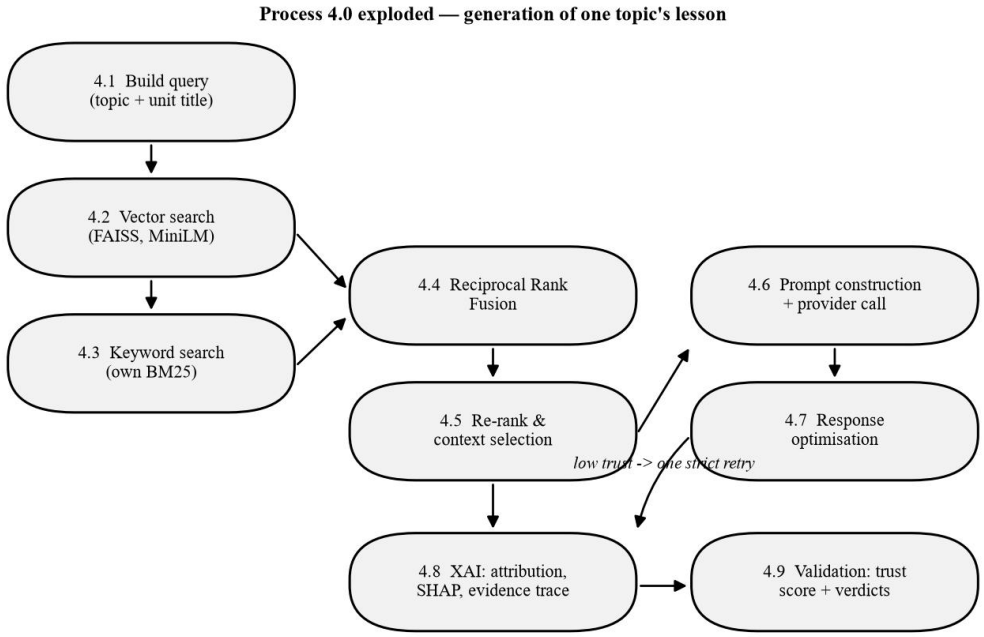


Fig 4.4: DFD Level 2 - retrieval, generation, explanation and validation

4.5 Entity-Relationship Model

The persistent model comprises six entities. A syllabus owns units, a unit owns topics and a topic owns subtopics, each as an identifying one-to-many relationship with cascade deletion. Reference sources are associated with a subject identifier, which is a string embedded in the topic key rather than a foreign key, because subjects are a structural level inside the parsed curriculum and are not materialised as their own table. That decision is examined in Section 7.4.

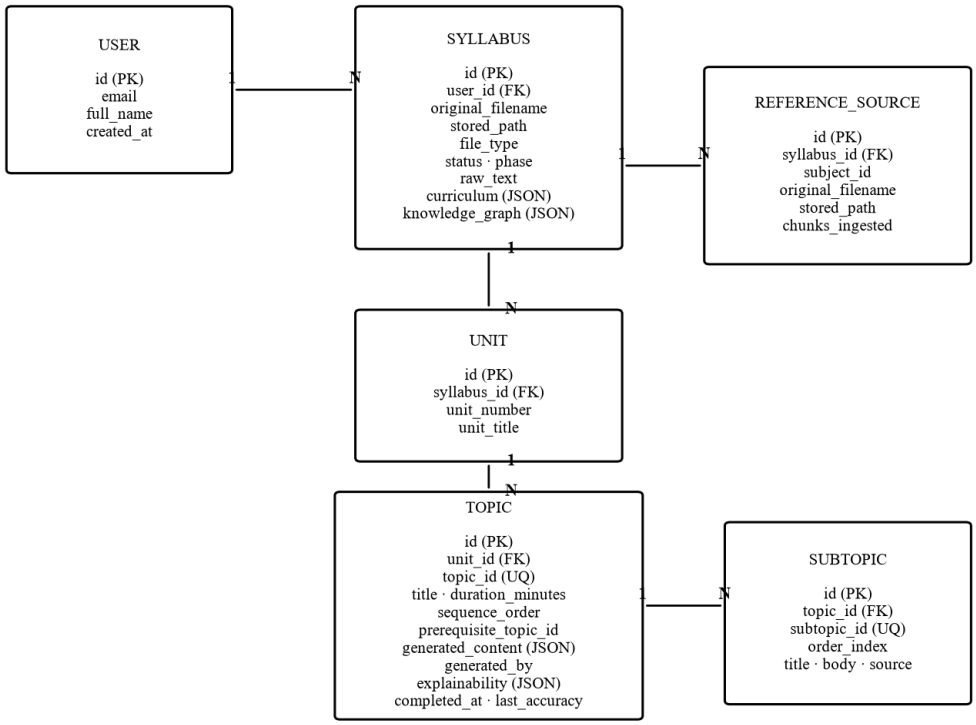


Fig 4.5: Entity-relationship diagram

4.6 Use-Case Model

Figure 4.6 shows the use cases supported by the implemented system. The learner is the only human actor; the language-model provider participates as a secondary actor in lesson generation only.

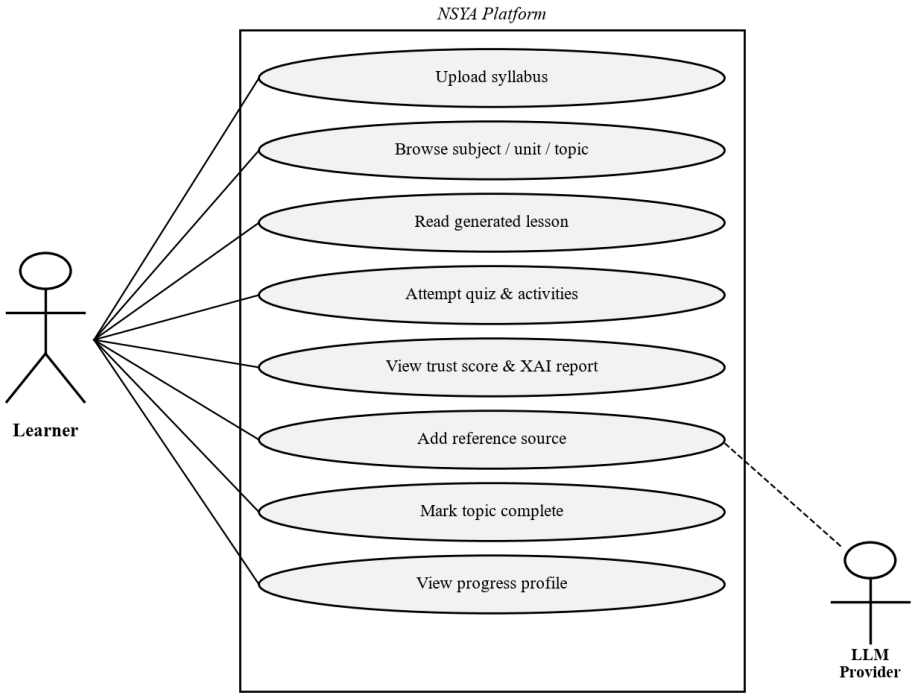


Fig 4.6: Use-case diagram

Table 4.2: Use-case specifications

Use case	Actor	Precondition	Main outcome
Upload syllabus	Learner	A supported file is available	Curriculum extracted, graph built, indexes populated
Browse learning path	Learner	An upload has completed	Subjects, units and topics presented in sequence
Read lesson	Learner	A topic is selected	Lesson generated on first access, thereafter served from cache
Attempt activities	Learner	A lesson exists	Four activity types scored, accuracy computed
Inspect trust report	Learner	A lesson exists	Trust score, metrics, evidence trace, attribution and Shapley values displayed
Add reference source	Learner	A subject exists and holds fewer than two sources	Document indexed into that subject's vector index
Mark topic complete	Learner	A lesson has been worked through	Completion and accuracy persisted; next topic pre-generated
View progress	Learner	At least one topic completed	Aggregate completion and average accuracy displayed

4.7 API Design

The HTTP interface comprises fifteen endpoints. The design is resource-oriented: the upload creates a job whose identifier is the syllabus identifier, and all subsequent resources are addressed by the stable topic or subtopic key rather than by database row identifiers, so that the front end can construct URLs from the curriculum structure alone.

Table 4.3: HTTP API endpoints

Method	Path	Purpose
GET	/health	Liveness probe
POST	/syllabus/upload	Multipart upload; returns the job identifier
GET	/jobs/{job_id}	Processing status, phase and progress; returns the full subject tree with each topic's stored completion and accuracy once done
GET	/topics/{topic_id}	Lesson content; generates on first access. The regenerate flag bypasses the cache
GET	/topics/{topic_id}/subtopics	Lightweight list of a topic's subtopic sections
GET	/subtopics/{subtopic_id}	A single subtopic with its parent context
GET	/topics/{topic_id}/explainability	The complete Phase 4 and Phase 5 report
POST	/topics/{topic_id}/complete	Records completion and accuracy; triggers pre-generation of the next topic
POST	/subjects/{subject_id}/sources	Uploads a reference document (maximum two)
GET	/subjects/{subject_id}/sources	Lists a subject's reference documents
GET	/	Lists uploaded syllabi
GET	/{syllabus_id}	Retrieves one syllabus record
GET	/api/v1/knowledge/{syllabus_id}/graph	Knowledge graph as node-link JSON
GET	/api/v1/knowledge/{syllabus_id}/roadmap/{subject_id}	Topologically ordered learning path for a subject
POST	/api/v1/knowledge/ingest	Manual chunk-and-index of supplied text
POST	/api/v1/knowledge/search	Manual vector search within a subject's index

Two design details are worth noting. The health endpoint is registered before the syllabus router because that router exposes a catch-all path parameter; registration order determines matching, so the reverse order would cause the health check to be interpreted as a syllabus identifier. Second, the topic endpoint deliberately returns a renderable fallback rather than

a 404 when a topic row does not yet exist, so that a request arriving before background processing has committed its rows does not leave the interface stuck.

CHAPTER V
METHODOLOGY AND ALGORITHMS

5.1 Methodological Overview

The system implements a five-phase trustworthy retrieval-augmented generation pipeline. Phase 1 executes once per upload; Phases 2 to 5 execute once per topic, on first access, and their combined output is cached. Figure 5.1 summarises the phases and the modules that implement them.

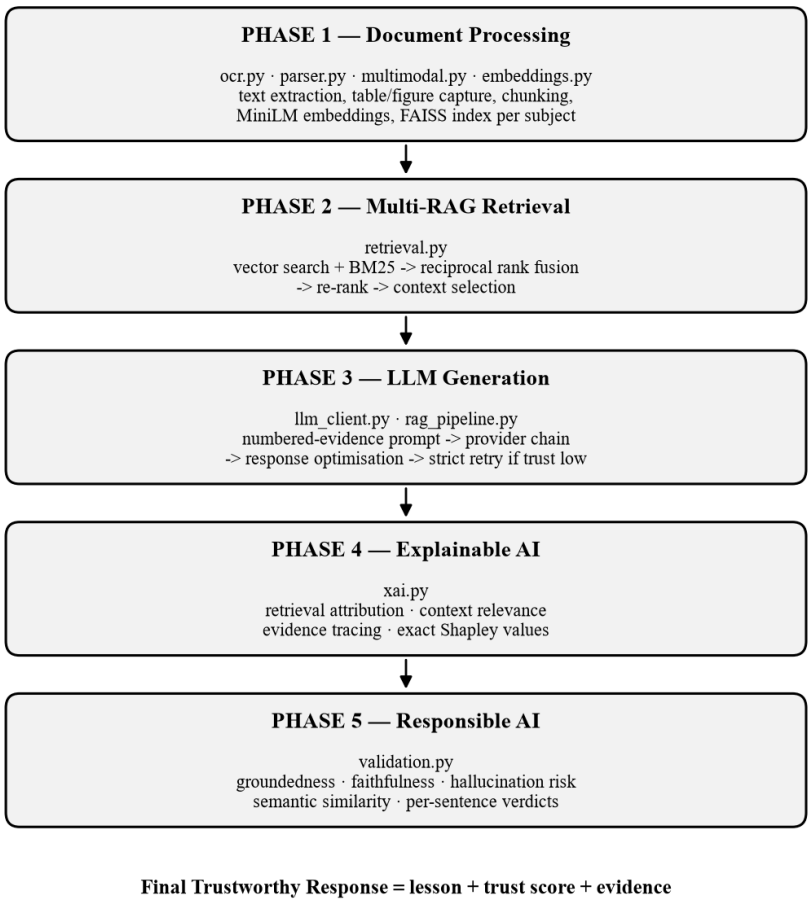


Fig 5.1: The five-phase processing pipeline

5.2 Phase 1 - Document Processing

5.2.1 Text Extraction

Extraction proceeds through ordered strategies and stops at the first that yields sufficient text. PyMuPDF is attempted first because it is fastest on documents with a text layer. If it returns fewer than eighty characters the document is assumed not to have a usable text layer and pdfplumber is attempted, which handles some layouts better. If that also fails, table extraction is attempted, and finally the pages are rasterised and passed to OCR if the optional OCR package is installed. Where OCR is required but unavailable, the system raises an error naming the installation script rather than returning an empty document.

Each strategy accumulates into its own buffer. This detail matters: an earlier revision shared

one buffer across strategies, so a PyMuPDF pass that produced some text but fell below the threshold left that text in place for pdfplumber to append to, duplicating every page in the extracted document. Because the extracted text is subsequently chunked and indexed, the duplication propagated into the retrieval evidence.

5.2.2 Curriculum Parsing

Parsing converts raw text into the subject-unit-topic structure. The document is first tested for multiple subjects by searching for explicit subject markers such as "Subject:" or "Course 2:". A split is performed only when at least two such markers are present, because unit-level words like "Chapter" legitimately recur within a single-subject syllabus and are too weak a signal to split on.

Within each subject, unit headings are matched against a pattern covering Unit, Chapter, Module and Part, with arabic or roman numbering. Topics are then extracted from each unit body by three strategies in order of preference: explicit list markers, then short heading-like lines, then noun-chunk extraction as a last resort. The minimum length accepted for an explicitly marked list item is deliberately lower than for the heading-based fallback, because a bullet already signals list membership and short but valid topics such as "Arrays" should not be discarded for brevity.

Each subject receives a derived identifier: an explicit code in the title is used if present, otherwise an acronym of the significant words with a default level suffix, giving for example CS101 for "Computer Science". Topic identifiers are then composed as subject-unit-topic, for example CS101-U1-T1. Because that identifier is globally unique, a second upload producing the same subject title would collide with the first; the system therefore checks for an existing identifier belonging to a different syllabus before creating rows and, on collision, rewrites the subject identifier and all dependent topic identifiers for the new upload.

5.2.3 Multimodal Extraction

Tables and figures carry syllabus information that a text-layer extraction discards. Tables are located with pdfplumber and linearised into sentences of the form "Row 1: Unit = 1; Topic = Arrays; Hours = 4.", which places the column headings next to their values so that the row remains meaningful to a text retriever. Embedded images are extracted, filtered by minimum dimension to exclude bullets and rules, and described by their measured properties: orientation, dominant colours, edge density and a classification into chart or diagram, photograph, scanned text or icon.

That classification is a heuristic over pixel statistics - colour variety, the share of near-white background, and the proportion of saturated non-background pixels - and not a trained classifier. It is labelled as a heuristic wherever its output is shown. Optional neural captioning is supported but produces a caption only when the model is actually available; when it is not, the figure receives its measured description and no fabricated caption. The resulting table and figure text is appended to the document under a marker line so that it is chunked and indexed along with the prose, allowing a later retrieval to label evidence by modality.

5.2.4 Chunking and Indexing

Chunking is paragraph-aware rather than fixed-width. Paragraphs are accumulated up to a target size of 400 characters, and only a paragraph that exceeds the target on its own is split by character count with a 50-character overlap. The rationale is that a chunk spanning two topics scores poorly for both when matched against a topic query, so preserving paragraph

boundaries improves retrieval precision.

Each chunk is embedded with the all-MiniLM-L6-v2 model into a 384-dimensional vector, normalised, and added to the subject's FAISS inner-product index, which is equivalent to cosine similarity for normalised vectors. The chunk text itself is always stored in a parallel metadata list, independently of whether embedding succeeded. That separation is what allows lexical retrieval to continue to function on a machine where the embedding model cannot be loaded.

5.3 Phase 2 - Multi-Strategy Retrieval

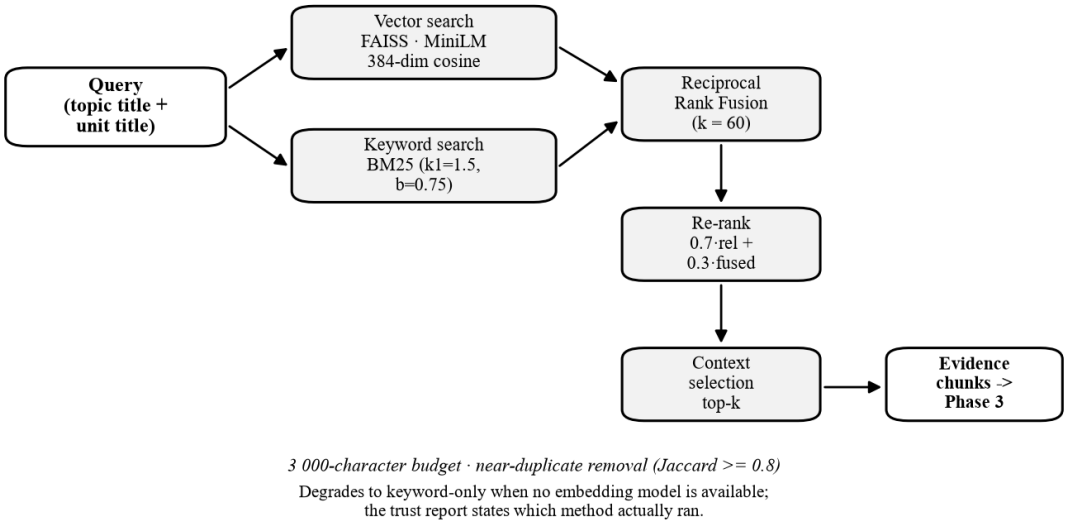


Fig 5.2: Hybrid retrieval and context selection

5.3.1 Dense and Lexical Retrieval

The query is the concatenation of the topic and unit titles. Dense retrieval embeds the query and searches the subject index for the nearest chunks by cosine similarity. Lexical retrieval scores every chunk with BM25:

```
score(D, Q) = SUM over terms t in Q of
                IDF(t) * ( f(t,D) * (k1 + 1) )
                       / ( f(t,D) + k1 * (1 - b + b * |D| / avgdl) )

        IDF(t) = ln( 1 + (N - df(t) + 0.5) / (df(t) + 0.5) )
        k1 = 1.5,  b = 0.75
```

The two retrievers fail differently, which is the reason for using both: dense retrieval handles paraphrase but can miss an exact technical term, while BM25 matches the term exactly but is blind to synonymy. BM25 is also the component that keeps the system usable when no embedding model is present.

5.3.2 Reciprocal Rank Fusion

The two ranked lists are combined by reciprocal rank fusion, which uses only rank position and therefore needs no score normalisation between two incomparable scales:

```
RRF(d) = SUM over rankings r of  1 / (k + rank_r(d)),    k = 60
```

A document ranked highly by both retrievers accumulates contributions from both and rises above a document favoured by only one, which is the intended behaviour.

5.3.3 Re-ranking and Context Selection

Fusion establishes agreement but discards magnitude, so a re-ranking step restores it. An absolute relevance value is computed for each candidate - a weighted blend of vector similarity and query-term coverage where a vector score exists, and coverage alone otherwise - and blended with the normalised fusion score:

```
relevance = 0.6 * vector_similarity + 0.4 * query_coverage   (if vector available)
relevance = query_coverage                                   (otherwise)
rerank    = 0.7 * relevance + 0.3 * (fused / max_fused)
```

Context selection then walks the re-ranked list and accepts chunks until either the requested count or a 3,000-character budget is reached, skipping any candidate whose token Jaccard similarity with an already-selected chunk is 0.8 or above. Near-duplicate suppression matters because overlapping chunks would otherwise consume the budget with redundant evidence and distort the Shapley analysis, in which two identical chunks each appear to be individually dispensable.

Every stage records what it did - whether each retriever ran, how many candidates were fused, whether re-ranking occurred and how many chunks were selected - and that trace is returned with the evidence so that Phase 4 can report the retrieval method honestly as hybrid, vector-only, keyword-only or excerpt fallback.

5.4 Phase 3 - Generation and Response Optimisation

5.4.1 Prompt Construction

The prompt states the subject, unit and topic, the prerequisite topic if one exists and the position in the learning path, and then presents the selected evidence as a numbered list. The model is instructed to ground the lesson in that evidence, to decompose the topic into as many subtopics as genuinely exist rather than a fixed number, and to return a single JSON object containing the lesson sections and the four activity types. When no evidence was retrieved, the prompt falls back to supplying a raw excerpt and states explicitly that outside knowledge may be used to fill gaps, so that the generation mode is never ambiguous.

5.4.2 Provider Chain and Diagnostics

Providers are attempted in a configured order. For each, the system records the outcome: success, absence of an API key, an HTTP error with the provider's own message, an unparseable response, or a response whose JSON lacks the required fields. Recording the reason is a deliberate correction of an earlier behaviour in which every failure returned the same empty result, making a rejected key, a retired model identifier and an unconfigured system indistinguishable from one another in the interface.

Responses are parsed leniently, because models commonly wrap JSON in code fences or prefix it with conversational text despite instructions; the parser extracts the outermost JSON object and validates that the required keys are present before accepting it.

5.4.3 Template Fallback

When no provider produces a valid response the system generates a template lesson. The template is explicitly structural: it describes the topic's position in the unit, its prerequisite and how to approach it, and these blocks are marked as scaffolding so that the Phase 5 validators exclude them - they make no claim about the subject, so scoring them would inflate the result. Where evidence was retrieved, the template additionally emits a section containing sentences copied verbatim from that evidence, with the source named. Verbatim

extraction is faithful by construction, which is why a template lesson with evidence can legitimately attain a high trust score while a template lesson without evidence is reported as unscorable.

5.4.4 Response Optimisation and Strict Retry

The response is normalised before validation: empty or duplicate sections are removed, whitespace is collapsed, and a quiz item whose answer index falls outside its option list is clamped with a recorded note. If the validated trust score then falls below 0.45 and the lesson came from a real provider, one further generation is attempted with a stricter instruction forbidding claims absent from the evidence. The retry is bounded to a single attempt, both drafts are scored, and the higher-scoring draft is retained; the report states which was kept.

5.5 Phase 4 - Explainable AI

5.5.1 Retrieval Attribution and Context Relevance

Attribution apportions the grounding across source documents by normalising the relevance scores of the retrieved chunks by their source, answering the question of which uploaded document the lesson actually drew on. Context relevance reports, for each chunk, its relevance, its separate vector and keyword components, its rank, the number of generated sentences it supports and its Shapley share.

5.5.2 Shapley Values over Retrieved Evidence

The contribution of each evidence chunk is quantified with Shapley values [2]. The characteristic function is defined over subsets of the evidence set as the mean, across generated sentences, of the best similarity to any chunk in the subset:

```
v(S)   = mean over sentences i of   max over chunks j in S of  sim(i, j)
v({})  = 0

phi_i  = SUM over S subset of N\{i} of
             [ |S|! * (n - |S| - 1)! / n! ] * ( v(S + {i}) - v(S) )
```

This measures how much each retrieved chunk contributes to the lesson's groundedness. Because v is monotone and v({}) = 0, the values are non-negative and sum exactly to the groundedness of the full evidence set, which is the efficiency axiom and provides a direct correctness check. The implementation enumerates all subsets exactly when the evidence set contains at most ten chunks, memoising coalition values, and switches to permutation sampling beyond that bound to keep the cost tractable.

The scope of this explanation should be stated precisely. It explains the dependence of the lesson's groundedness on the retrieved evidence. It is not an explanation of the language model's internal computation, which is not observable because the model is accessed over HTTP. The implementation is also a native one written against the Shapley definition rather than a use of an existing attribution library.

5.5.3 Evidence Tracing

Evidence tracing links each generated sentence to the chunk that best supports it, carrying the similarity value, the verdict from Phase 5, the source document and the modality of the evidence - text, table or figure. This is the output a learner uses directly: it answers, sentence by sentence, where a statement came from.

5.6 Phase 5 - Responsible AI Validation

5.6.1 Evidence Construction and the Similarity Backend

The lesson is segmented into sentences, excluding scaffolding blocks, and a similarity matrix of sentences against evidence chunks is computed once and reused by every subsequent metric. The similarity backend prefers sentence embeddings and falls back to TF-IDF cosine similarity over stemmed, stop-word-filtered tokens when the embedding model cannot be loaded. Because the two methods produce different score distributions - a paraphrase scores far lower under TF-IDF than under embeddings - the support thresholds are calibrated separately for each, and the method actually used is reported in the output so that a score is never interpreted against the wrong scale.

Table 5.1: Similarity thresholds calibrated per method

Parameter	Embedding method	Lexical (TF-IDF) method
Support threshold	0.45	0.22
Weak threshold	0.30	0.10
Normalisation target	0.75	0.45

5.6.2 Validation Metrics

Four metrics are computed from the similarity matrix.

```
best_i             = max over chunks j of sim(i, j)      for sentence i
groundedness       = mean over i of best_i
faithfulness       = |{ i : best_i >= support }| / n
hallucination_risk = ( unsupported + 0.5 * weak ) / n
semantic_similarity= sim( whole lesson , whole retrieved context )
```

Each sentence is additionally classified: supported where its best similarity reaches the support threshold, weak where it reaches only the weak threshold, and unsupported below that. A numeric check runs alongside - a sentence containing a number that appears nowhere in the evidence is downgraded from supported to weak, with the specific value recorded as the reason, because fabricated figures are both a common failure of generated text and easy to verify mechanically.

5.6.3 Composite Trust Score

The five normalised components are combined under fixed published weights, which are returned with every report so that the score can be recomputed and audited:

Table 5.2: Trust score components and weights

Component	Weight	Definition
Groundedness	0.30	Mean best-evidence similarity, normalised by the method target
Faithfulness	0.25	Proportion of sentences meeting the support threshold
Low hallucination	0.20	1 - hallucination risk
Semantic similarity	0.15	Whole-lesson against whole-context, normalised
Retrieval confidence	0.10	Mean relevance of the retrieved chunks

The weighted sum is mapped to a band: high at 0.70 and above, medium at 0.45 and above, low below that. Where no evidence exists, or where the lesson makes no validatable claim, the score is reported as unavailable together with the reason. This case is treated as a requirement rather than an edge case: returning zero would be indistinguishable from a lesson that was assessed and found entirely unsupported, which is a materially different

situation.

5.7 Knowledge Graph Construction and Sequencing

The curriculum is converted into a directed graph whose nodes are subjects, units and topics. Part-of edges connect topics to units and units to subjects. Prerequisite edges chain units in order and, importantly, chain topics across the whole subject rather than within each unit independently. Without the cross-unit chain the topological sort has no ordering constraint between the last topic of one unit and the first of the next, so topics from a later unit can be ordered ahead of earlier ones. A topological sort over the subject's subgraph then yields the learning order, from which each topic's sequence number and immediate prerequisite are recorded. A cycle, which the construction rules should make impossible, is caught and handled by falling back to a deterministic unit-then-topic ordering rather than failing the upload.

CHAPTER VI
IMPLEMENTATION AND MODULE DESCRIPTION

6.1 Implementation Overview

The system comprises 72 source files totalling approximately 12,900 lines. The distribution across the layers is given in Table 6.1. The backend is Python; the front end is JavaScript with no framework, no package manager and no build step, so it is served as static files exactly as written.

Table 6.1: Source code distribution

Area	Files	Lines	Principal content
Backend - application core	4	204	Entry point, settings, response schemas
Backend - API routers	4	1,209	Fifteen HTTP endpoints across two routers
Backend - database layer	3	146	Declarative models, engine and session handling
Backend - services	13	2,385	The five-phase pipeline and its support modules
Backend - test suite	9	1,590	65 automated tests and shared fixtures
Front end - markup and styles	2	2,473	Single page shell and the full style sheet
Front end - core modules	11	1,510	Router, store, API client, report renderer, panels
Front end - screens	8	1,329	Upload, progress, subjects, units, topics, lesson, profile
Front end - activities	6	591	Four activity types and their shared chrome
Front end - test harness	2	488	DOM stub and 13-check smoke suite
Project tooling	10	988	Installers, launchers and operational scripts

6.2 Module Interaction

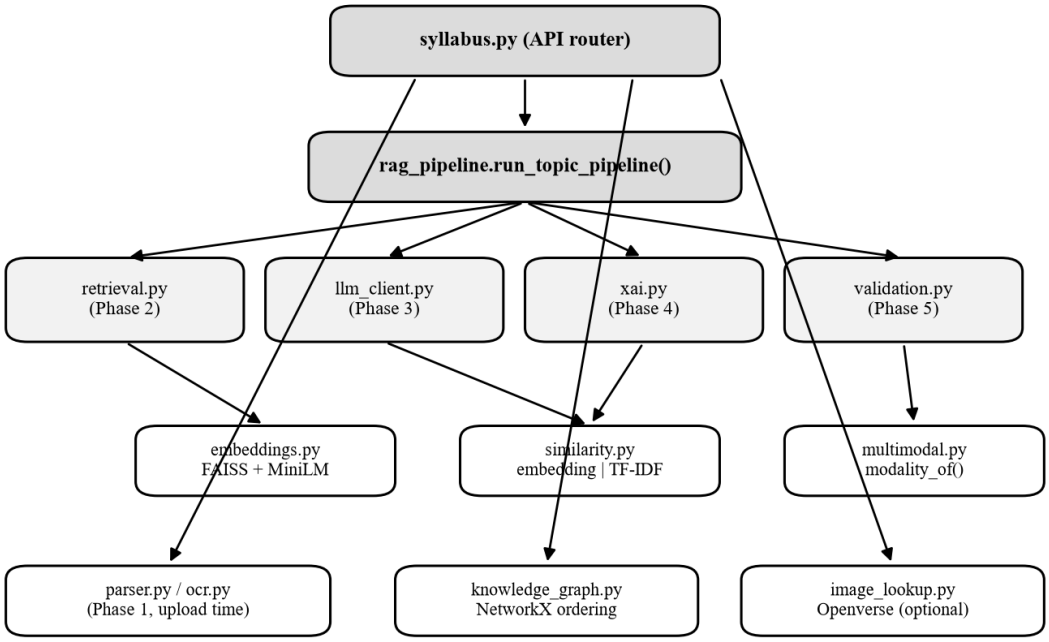


Fig 6.1: Module interaction map

Figure 6.1 shows the call direction between modules. The API router invokes the pipeline

orchestrator, which calls each phase module in turn; the phase modules share the similarity backend and the embedding store but do not call one another directly. Keeping the phases mutually independent is what allows each to be tested in isolation with the others stubbed.

6.3 Backend Module Specifications

Each principal module is specified below in terms of its purpose, inputs, processing, outputs, dependencies, error handling and implementation status.

6.3.1 Document Extraction

Table 6.2: Module specification - Document Extraction

Attribute	Specification
Source file	app/services/ocr.py
Purpose	Converts an uploaded file into plain text.
Input	Absolute file path and a declared file type of pdf, image or text.
Processing	For text files, direct read. For PDFs, PyMuPDF first, then pdfplumber, then table extraction, then rasterisation with OCR; each strategy uses its own buffer and the first producing more than eighty characters wins. For images, OCR directly.
Output	Extracted text, with the multimodal appendix appended for PDFs.
Dependencies	PyMuPDF, pdfplumber, optional PaddleOCR.
Error handling	A missing file raises immediately. Each strategy is individually guarded so that one failure advances to the next. Where OCR is required but not installed, a message naming the installation script is raised rather than returning empty text.
Status	Implemented and tested

6.3.2 Curriculum Parser

Table 6.3: Module specification - Curriculum Parser

Attribute	Specification
Source file	app/services/parser.py
Purpose	Derives the subject-unit-topic hierarchy from raw text.
Input	Raw extracted text.
Processing	Detects multiple subjects by explicit markers, splitting only when at least two are present; matches unit headings for Unit, Chapter, Module and Part in arabic or roman numerals; extracts topics by list markers, then heading-like lines, then noun chunks; derives subject identifiers and estimates durations.
Output	A nested curriculum dictionary of subjects, units and topics, each topic carrying a unique identifier, title and estimated duration.
Dependencies	spaCy, with a blank pipeline plus sentencizer as fallback.
Error handling	An empty document yields a valid empty curriculum rather than an exception. A unit with no detectable topics is retained so the structure is not silently lost. Noun-chunk extraction is guarded because it requires a model that may be absent.
Status	Implemented and tested

6.3.3 Multimodal Extraction

Table 6.4: Module specification - Multimodal Extraction

Attribute	Specification
Source file	app/services/multimodal.py
Purpose	Recovers tables and figures from PDFs as retrievable text.
Input	PDF file path.
Processing	Extracts tables with pdfplumber and linearises each row with its column headings; extracts embedded images, filters by minimum dimension, computes colour and edge statistics, classifies the image type heuristically and composes a description; optional neural captioning is attempted only if the model loads.

Attribute	Specification
Output	An appendix of table and figure text, plus structured metadata for each.
Dependencies	pdfplumber, PyMuPDF, Pillow, NumPy; optional transformers.
Error handling	Every stage is individually guarded; failure returns an empty appendix so that plain text extraction is never broken by a malformed embedded object. Captioning returns nothing rather than a fabricated caption when the model is unavailable.
Status	Implemented and tested

6.3.4 Embeddings and Vector Store

Table 6.5: Module specification - Embeddings and Vector Store

Attribute	Specification
Source file	app/services/embeddings.py
Purpose	Chunks text, generates embeddings and maintains the per-subject index.
Input	Subject identifier, topic identifier, document text, source label.
Processing	Paragraph-aware chunking to a 400-character target with 50-character overlap for oversized paragraphs; embedding to 384 dimensions with normalisation; addition to a FAISS inner-product index with a parallel metadata record per chunk.
Output	Number of chunks indexed; search returns chunks with scores and metadata.
Dependencies	faiss-cpu, sentence-transformers, NumPy.
Error handling	Chunk text is stored even when embedding fails, preserving lexical retrieval. A failure to load the model is latched so that subsequent calls fail immediately instead of repeating an expensive retry, and the failure is raised as a typed exception that callers treat as a signal to degrade.
Status	Implemented and tested

6.3.5 Hybrid Retrieval

Table 6.6: Module specification - Hybrid Retrieval

Attribute	Specification
Source file	app/services/retrieval.py
Purpose	Selects the evidence set for a topic.
Input	Subject identifier, query string, requested chunk count.
Processing	Runs vector search and BM25, fuses by reciprocal rank fusion, computes an absolute relevance per candidate, re-ranks, and selects under a character budget with near-duplicate suppression.
Output	Selected chunks with per-chunk scores, and a trace recording which retrievers ran and how many candidates were considered.
Dependencies	The embedding store and the shared tokeniser.
Error handling	An absent or empty index returns no chunks and a trace recording that, rather than an error. Vector search failure degrades to keyword-only and is recorded in the trace. Where no index exists at all, the caller falls back to ranking the raw document text.
Status	Implemented and tested

6.3.6 Similarity Backend

Table 6.7: Module specification - Similarity Backend

Attribute	Specification
Source file	app/services/similarity.py
Purpose	Provides the similarity measure shared by Phases 2, 4 and 5.
Input	Two sequences of strings.
Processing	Attempts sentence embeddings; on failure computes TF-IDF cosine similarity over stemmed, stop-word-filtered tokens.
Output	A similarity matrix in the range zero to one, and the name of the method used.

Attribute	Specification
Dependencies	The embedding store; NumPy.
Error handling	A load failure is latched so the fallback is used consistently for the remainder of the process rather than retried per call. The method name is always returned so callers can select the correct thresholds.
Status	Implemented and tested

6.3.7 Language Model Client

Table 6.8: Module specification - Language Model Client

Attribute	Specification
Source file	app/services/llm_client.py
Purpose	Constructs the prompt and obtains a lesson from a provider.
Input	Topic, unit and subject titles, prerequisite, sequence position, evidence chunks, and a strict-mode flag.
Processing	Builds a prompt presenting evidence as numbered references; attempts each configured provider in order; extracts JSON from the response, tolerating code fences and conversational prefixes; validates the required keys.
Output	A validated lesson dictionary with the provider recorded, or nothing if all providers fail; a per-provider outcome list in both cases.
Dependencies	Standard library HTTP only, so no additional dependency is introduced.
Error handling	Each provider call is individually guarded. Failures are classified and recorded - missing key, HTTP status with the provider's message, unreachable host, unparseable response, or invalid shape - and the list is surfaced in the report so a misconfiguration is visible rather than appearing as an unconfigured system.
Status	Implemented and tested

6.3.8 Pipeline Orchestrator

Table 6.9: Module specification - Pipeline Orchestrator

Attribute	Specification
Source file	app/services/rag_pipeline.py
Purpose	Executes Phases 2 to 5 for one topic and assembles the report.
Input	Topic context, the raw document text, the subject identifier and the generation and template callables.
Processing	Retrieves evidence, falling back to ranking the raw text if no index exists; generates; optimises the response; runs the explainability and validation analyses; performs one strict regeneration if the trust score is low and a real provider was used; assembles the final report.
Output	The lesson body and the complete explainability report.
Dependencies	The retrieval, generation, explainability and validation modules.
Error handling	Retrieval failure degrades to the excerpt fallback. Generation failure selects the template. A regeneration that does not improve the score is discarded and the decision is recorded.
Status	Implemented and tested

6.3.9 Explainable AI

Table 6.10: Module specification - Explainable AI

Attribute	Specification
Source file	app/services/xai.py
Purpose	Computes the Phase 4 analyses.
Input	The evidence object, the per-sentence verdicts and the retrieved chunks.
Processing	Normalises relevance by source for attribution; computes exact Shapley values with memoised coalition values, switching to permutation sampling above ten chunks; builds the per-sentence evidence trace; assembles the per-chunk relevance report.

Attribute	Specification
Output	Attribution weights, Shapley values with shares, the evidence trace and the context relevance report.
Dependencies	NumPy; the validation module's evidence structure.
Error handling	An absent evidence set returns an explicitly unavailable result rather than empty structures that would read as a computed zero.
Status	Implemented and tested

6.3.10 Responsible AI Validation

Table 6.11: Module specification - Responsible AI Validation

Attribute	Specification
Source file	app/services/validation.py
Purpose	Computes the Phase 5 metrics and the trust score.
Input	Lesson sections and the retrieved chunks.
Processing	Segments sentences, excluding scaffolding; computes the similarity matrix once; derives groundedness, faithfulness, hallucination risk and semantic similarity; classifies each sentence and applies the numeric check; combines the normalised components under the published weights and assigns a band.
Output	The full metric set, per-sentence verdicts, flagged sentences, the trust score and band, and the components and weights used.
Dependencies	The similarity backend; NumPy.
Error handling	Where there is no evidence or no validatable claim, every metric is returned as unavailable with a stated reason instead of a default value.
Status	Implemented and tested

6.3.11 Knowledge Graph

Table 6.12: Module specification - Knowledge Graph

Attribute	Specification
Source file	app/services/knowledge_graph.py
Purpose	Builds the prerequisite graph and derives the learning order.
Input	The parsed curriculum.
Processing	Creates subject, unit and topic nodes; adds part-of edges and prerequisite edges chaining units and chaining topics across the whole subject; topologically sorts the subject subgraph.
Output	A serialisable graph and an ordered roadmap giving each topic its position and prerequisite.
Dependencies	NetworkX.
Error handling	A cycle is caught and handled by a deterministic fallback ordering rather than failing the upload.
Status	Implemented and tested

6.3.12 API Routers

Table 6.13: Module specification - API Routers

Attribute	Specification
Source file	app/api/routers/
Purpose	Exposes the system over HTTP.
Input	HTTP requests.
Processing	Validates and dispatches to the services; manages the upload background task and the processing phases; caches generated lessons; enforces the reference-source limit; records completion.
Output	JSON responses as specified in Table 4.3.

Attribute	Specification
Dependencies	FastAPI, SQLAlchemy, the service layer.
Error handling	The upload task records a failure state with a message rather than leaving a job pending, rolling back the session first so that the failure can itself be committed. A per-topic lock ensures concurrent requests produce one generation. Unsupported uploads are rejected with a readable message.
Status	Implemented and tested

6.4 Front-End Implementation

The front end is deliberately framework-free. Routing is performed by a hash router, so the application can be served from any static file server without rewrite rules. Application state is held in a small publish-subscribe store persisted to browser storage, so a refresh does not lose the learner's position or progress. Rendering is direct DOM construction through a single helper, with each screen redrawing the region that changed.

Table 6.14: Front-end modules

Module	Responsibility
main.js, router.js	Route registration and hash-based navigation
store.js	Pub-sub state container persisted to browser storage, with merge semantics that never clear local progress
api.js	Typed client for all endpoints, with upload progress via XHR
job-sync.js	Polls a running job; re-reads a finished job once per mount so server-stored progress is picked up
shell.js	Shared navigation rail and page frame
screens/	Upload, uploading, extracting, subjects, units, topics, lesson and profile
games/	Four activity types sharing common chrome
xai-metrics.js	The single renderer for the Phase 4 and 5 report, used by both the lesson popup and the unit drawer
xai-panel.js, sources-panel.js	Per-unit explainability drawer and per-subject reference-source upload

The report renderer deserves specific mention. Both the per-lesson popup and the per-unit drawer display the same Phase 4 and 5 report, and an earlier design rendered each separately. Consolidating them into one renderer removed the possibility of the two views disagreeing about the same lesson. The renderer draws a bar for every metric the backend could compute and an explicit "N/A" for any it could not, so an unavailable metric is visibly distinct from a low one.

6.5 Representative Implementation Excerpts

The following excerpts are reproduced from the implementation to illustrate the two computations most central to the project's contribution.

Exact Shapley values over the retrieved evidence (app/services/xai.py):

```
def shapley_values(sim):
    n = sim.shape[1]
    if n <= EXACT_LIMIT:                        # exact enumeration
        cache = {}
        def v(cols):
            if cols not in cache:
                cache[cols] = _value(sim, list(cols))
            return cache[cols]
        phi = [0.0] * n
        for i in range(n):
```

```
            others = [j for j in range(n) if j != i]
            for size in range(n):
                weight = fact(size) * fact(n - size - 1) / fact(n)
                for subset in itertools.combinations(others, size):
                    phi[i] += weight * (v(tuple(sorted(subset + (i,)))) -
v(subset))
        return phi
    ...                                          # permutation sampling beyond the
limit
```

Composite trust score (app/services/validation.py):

```
components = {
    'groundedness':         min(1.0, groundedness / target),
    'faithfulness':         faithfulness,
    'low_hallucination':    1.0 - hallucination_risk,
    'semantic_similarity':  min(1.0, semantic_similarity / target),
    'retrieval_confidence': retrieval_conf,
}
trust = sum(TRUST_WEIGHTS[k] * components[k] for k in TRUST_WEIGHTS)
band  = next(label for floor, label in TRUST_BANDS if trust >= floor)
```

CHAPTER VII
DATABASE AND DATA STRUCTURES

7.1 Database Technology

Persistence uses SQLite accessed through SQLAlchemy's declarative ORM. SQLite was selected because it requires no server process, which is a direct consequence of the operational requirement that a learner be able to run the system with one command. The ORM layer means the engine can be changed by altering one configuration value; the connection arguments and pooling parameters are already branched on the database type so that a PostgreSQL URL receives pool sizing while SQLite receives the thread-check setting it needs.

7.2 Schema Specification

The schema comprises six tables. Primary keys are string UUIDs generated at insertion, which allows a row to be referenced before the transaction commits and avoids depending on a database-generated sequence.

Table 7.1: users table

Field	Type	Constraint	Description
id	String(36)	Primary key	UUID generated on insertion
email	String(255)	Unique, not null, indexed	Owner email address
full_name	String(255)	Nullable	Display name
created_at	DateTime	Default now	Record creation timestamp

The users table is present but unpopulated: the application is single-user, and the syllabus foreign key to it is nullable. It is retained because the multi-user extension described in Section 10.4 depends on it and removing it would require a migration later.

Table 7.2: syllabi table

Field	Type	Constraint	Description
id	String(36)	Primary key	UUID; also used as the externally visible job identifier
user_id	String(36)	Foreign key, nullable	References users.id
original_filename	String(512)	Not null	Name as uploaded, used as the source label
stored_path	String(1024)	Not null	Path of the stored copy
file_type	String(50)	Not null	pdf, image or text
status	String(50)	Default uploaded	uploaded, processing, parsed or failed
phase	String(20)	Default extract	extract, distribute, generate, done or error
raw_text	Text	Nullable	Extracted text including the multimodal appendix
curriculum	JSON	Nullable	Parsed subject-unit-topic tree
knowledge_graph	JSON	Nullable	Serialised prerequisite graph
error_message	Text	Nullable	Failure reason when status is failed
created_at, updated_at	DateTime	Defaults	Creation and modification timestamps

Table 7.3: units table

Field	Type	Constraint	Description
id	String(36)	Primary key	UUID
syllabus_id	String(36)	Foreign key, not null	Owning syllabus; cascade delete
unit_number	Integer	Not null	Ordinal within the subject
unit_title	String(512)	Not null	Unit heading as parsed
created_at	DateTime	Default now	Creation timestamp

Table 7.4: topics table

Field	Type	Constraint	Description
id	String(36)	Primary key	UUID
unit_id	String(36)	Foreign key, not null	Owning unit; cascade delete
topic_id	String(64)	Unique, not null	Stable external key, e.g. CS101-U1-T1
title	String(512)	Not null	Topic title as parsed
duration_minutes	Integer	Default 45	Heuristic estimate
sequence_order	Integer	Nullable	Position in the subject roadmap
prerequisite_topic_id	String(64)	Nullable	Immediate predecessor's external key
generated_content	JSON	Nullable	Cached activities, notes and optional image
generated_by	String(20)	Nullable	Provider that produced the lesson, or template
explainability	JSON	Nullable	Complete Phase 4 and 5 report
completed_at	DateTime	Nullable	Set when the learner marks the topic complete
last_accuracy	Integer	Nullable	Most recent accuracy, clamped to 0-100
created_at	DateTime	Default now	Creation timestamp

Table 7.5: subtopics table

Field	Type	Constraint	Description
id	String(36)	Primary key	UUID
topic_id	String(36)	Foreign key, not null	Owning topic; cascade delete
subtopic_id	String(96)	Unique, not null	Stable external key, e.g. CS101-U1-T1-S1
order_index	Integer	Not null, default 1	Position within the topic
title	String(512)	Not null	Section heading
body	Text	Nullable	Section text
source	String(255)	Nullable	Stated provenance of the section
created_at	DateTime	Default now	Creation timestamp

Table 7.6: reference_sources table

Field	Type	Constraint	Description
id	String(36)	Primary key	UUID
syllabus_id	String(36)	Foreign key, not null	Syllabus the subject belongs to
subject_id	String(64)	Not null	Subject key, e.g. CS101; not a foreign key
original_filename	String(512)	Not null	Name as uploaded
stored_path	String(1024)	Not null	Path of the stored copy
chunks_ingested	Integer	Not null, default 0	Chunks contributed to the index
created_at	DateTime	Default now	Creation timestamp

7.3 Relationships and Integrity

Table 7.7: Entity relationships and integrity rules

Relationship	Cardinality	Integrity rule
users to syllabi	1 : N	Optional; the foreign key is nullable in single-user operation
syllabi to units	1 : N	Identifying; deleting a syllabus cascades to its units
units to topics	1 : N	Identifying; deleting a unit cascades to its topics
topics to subtopics	1 : N	Identifying; cascade delete, and regeneration removes existing rows before inserting new ones so that duplicates cannot accumulate
syllabi to reference_sources	1 : N	Non-identifying; association to a subject is by the subject key string
topic to prerequisite topic	0..1 : 1	Self-reference by external key, derived from the knowledge graph

Uniqueness is enforced on topic_id and subtopic_id because these are the keys the front end constructs URLs from; they must therefore be stable and globally unique. That uniqueness is also the reason for the collision-resolution step described in Section 5.2.2: two independent uploads that happen to yield the same subject title would otherwise violate the constraint and abort the second upload.

7.4 Design Decisions and Trade-offs

- **Subjects are not a table.** Subjects exist as a level within the parsed curriculum JSON and as a prefix within the topic key, not as rows. This keeps the parsed structure authoritative and avoids synchronising two representations, at the cost that reference sources associate with a subject by string rather than by foreign key. The trade-off was accepted because subject identity is derived from parsing and can be rewritten during collision resolution.
- **Subtopics are rows, not embedded JSON.** Lesson sections are stored as first-class rows with their own stable identifiers, so they can be addressed, listed cheaply without loading full text, and regenerated independently. The alternative - an array inside the cached lesson - would have made the subtopic endpoints impossible to serve efficiently.
- **Reports are stored, not recomputed.** The explainability report is persisted with the topic. Recomputation would require the original evidence set, which may change as reference sources are added; storing the report preserves the evaluation exactly as it was performed for the lesson shown.
- **Progress is stored server-side as well as locally.** Completion and accuracy are held both in browser storage, for immediate rendering, and on the topic row. The two are merged on load in a direction that can only add progress, never clear it.

7.5 Non-Relational Data Structures

Table 7.8: Non-relational data structures

Structure	Representation	Purpose and location
Vector index	FAISS inner-product index over normalised 384-dimensional vectors	One directory per subject under the configured index path; provides dense retrieval

Structure	Representation	Purpose and location
Chunk metadata store	Pickled list of records, parallel to the index	Holds chunk text, source, topic key and the vector row identifier; the authoritative source of chunk text, which is why lexical retrieval survives the absence of embeddings
Term statistics	Per-chunk token frequency maps and a document-frequency counter	Constructed in memory per query to score BM25
Similarity matrix	NumPy array of shape (sentences x chunks)	Computed once per lesson and shared by the explainability and validation stages
Coalition cache	Dictionary keyed by sorted chunk-index tuples	Memoises the characteristic function during exact Shapley computation
Knowledge graph	NetworkX directed graph, persisted as node-link JSON	Prerequisite ordering; stored on the syllabus row so it need not be rebuilt
Curriculum tree	Nested JSON of subjects, units and topics	Authoritative parsed structure, mirrored into the relational tables
Generation locks	Dictionary of per-topic locks guarded by a mutex	Ensures concurrent requests for one ungenerated topic produce a single generation

CHAPTER VIII
TESTING AND VALIDATION

8.1 Testing Strategy

Testing was approached on the principle that a system whose purpose is to measure trustworthiness must itself be verifiable. Three properties were therefore required of the suite. It must run offline, so that no result depends on network availability. It must be deterministic, so that a test does not pass on one machine and fail on another for environmental reasons. And it must isolate itself completely from the application's own data, so that running the tests cannot corrupt a learner's database or retrieval index.

The mathematics of Phases 4 and 5 is tested against its formal definition rather than against recorded output. Shapley values are checked against a brute-force enumeration of the definition and against the efficiency, symmetry and dummy-player axioms; this is stronger than a regression test, because it would detect an error that was present from the first implementation.

Table 8.1: Levels of testing applied

Level	Scope	Representative coverage
Unit	Individual functions in isolation	BM25 ranking, reciprocal rank fusion, context selection, Shapley values, trust weights, sentence verdicts
Integration	Several modules together	Retrieval with and without embeddings, the full pipeline with a stubbed provider, template fallback with and without evidence
System	The whole application over real HTTP	Upload to completion, topic generation, explainability retrieval, completion recording
Regression	Previously fixed defects	Cross-unit ordering, duplicate-upload collision, failure recording, duplicate subtopics on regeneration
Concurrency	Parallel access to one resource	Five simultaneous requests for one ungenerated topic
Interface	Front-end behaviour	The complete lesson flow through all activity stages, driven in Node against a DOM stub

8.2 Test Environment and Isolation

The backend suite runs under pytest. Every test receives a freshly created database, an empty retrieval index directory and an empty upload directory, all redirected away from the application's own paths by environment variables read before the application modules are imported.

This isolation is not incidental. Indexing is keyed by subject identifier, and the test fixtures deliberately use realistic identifiers such as CS101. With a shared index directory, every test run wrote fixture chunks into the same index that a developer's running application retrieves from; thirty-six fixture chunks were found resident in the live index before the directory was made configurable. The upload directory had accumulated seventy-two fixture files for the same reason. Both are now redirected, and the isolation is itself verified after each run by confirming that the application's directories remain untouched.

Determinism is achieved by stubbing the two network-dependent components. The embedding model is either replaced by a deterministic hashed bag-of-words substitute or disabled entirely to exercise the lexical fallback, and the external image lookup is stubbed to return nothing by default. The latter corrected a test that asserted no image was found

and therefore passed only on a machine without internet access - it failed on any machine that had it, which is the opposite of a useful test.

8.3 Test Case Specifications

The table below specifies representative test cases across the modules. All listed cases were executed; the status column records the actual outcome of the final run.

Table 8.2: Test case specifications and outcomes

ID	Module	Test scenario	Input	Expected result	Status
TC-01	Parser	Single-subject syllabus parsed into correct units and topics	Sample syllabus, 2 units x 3 topics	Six topics in syllabus order with correct titles	Pass
TC-02	Parser	Multi-subject document split correctly	Two-subject sample	Two subjects, four units, three topics each	Pass
TC-03	Knowledge graph	Sequence order strictly increasing across unit boundaries	Parsed curriculum	Orders form 1..n with no gaps or repeats	Pass
TC-04	Knowledge graph	Prerequisite chains across a unit boundary	Parsed curriculum	First topic of unit 2 has last topic of unit 1 as prerequisite	Pass
TC-05	Upload pipeline	Duplicate upload does not collide	Same syllabus uploaded twice	Both succeed; topic identifier sets are disjoint	Pass
TC-06	Upload pipeline	Unrelated failure is recorded, not left pending	Syllabus row with a non-existent file path	Status failed with a stored error message	Pass
TC-07	Retrieval	Keyword-only retrieval when no embedding model is available	Indexed text, embeddings disabled	Relevant chunk returned; method reported as keyword-only	Pass
TC-08	Retrieval	Hybrid retrieval uses both retrievers	Indexed text, embeddings stubbed	Method reported as hybrid; vector score present; rank 1 correct	Pass
TC-09	Retrieval	Near-duplicate chunks suppressed in selection	Four candidates, two near-identical	Three selected, only one of the duplicate pair	Pass
TC-10	Retrieval	Empty index returns nothing rather than erroring	Unknown subject identifier	No chunks; method reported as none	Pass
TC-11	Explainable AI	Shapley values satisfy efficiency, symmetry and dummy player	Random similarity matrix with a duplicated and a null column	Sum equals total value; duplicates equal; null column zero	Pass
TC-12	Explainable AI	Shapley values match the brute-force definition	Random 5x3 matrix	Values equal the enumerated definition	Pass
TC-13	Explainable AI	Sampling path used beyond the exact limit	Matrix with 12 chunks	Correct length; sum matches total within tolerance	Pass
TC-14	Validation	Grounded lesson outscores off-topic lesson on every metric	Same evidence, two lessons	Groundedness, faithfulness and trust higher; hallucination risk lower	Pass
TC-15	Validation	Number absent from evidence downgrades a sentence	Lesson containing a fabricated figure	Verdict not supported; the specific value cited as the reason	Pass
TC-16	Validation	Trust weights sum to one	Weight table	Sum equals 1.0	Pass
TC-17	Validation	No evidence yields no score rather than zero	Lesson with empty evidence	available false; trust score null; reason stated	Pass
TC-18	Validation	Scaffolding blocks excluded from scoring	Template lesson with scaffold and evidence blocks	Only the evidence block contributes sentences	Pass
TC-19	Pipeline	Report contains every phase field	Stubbed provider, indexed evidence	All retrieval, explainability and validation fields present and non-null	Pass
TC-20	Pipeline	Low trust triggers exactly one strict regeneration	Provider returning off-topic then grounded content	Two calls, second strict; regenerated draft kept	Pass
TC-21	Generation	Subtopics persisted as real rows	Topic generation request	Row count equals section count; identifiers correctly formed	Pass
TC-22	Generation	Content cached, not regenerated on revisit	Two successive requests	Identical content returned	Pass
TC-23	Generation	Regeneration replaces rather	Forced regeneration	Row count unchanged	Pass

ID	Module	Test scenario	Input	Expected result	Status
		than duplicates subtopics			
TC-24	Generation	Image key absent when no image is found	Image lookup stubbed to none	Key absent entirely, not null or empty	Pass
TC-25	Generation	Image passed through intact when one is found	Image lookup stubbed to a result	Image object returned unchanged	Pass
TC-26	Concurrency	Concurrent requests produce one generation	Five parallel requests for one topic	Exactly one generation call; all callers receive content	Pass
TC-27	LLM providers	Request shape and response parsing per provider	Stubbed HTTP layer	Correct endpoint, authorisation header and body; response parsed	Pass
TC-28	LLM providers	Provider order honoured; malformed response falls through	First provider returns invalid JSON	Second provider's result used	Pass
TC-29	LLM providers	Rejected key reported, not swallowed	Provider returning HTTP 401	Outcome recorded with status and provider message	Pass
TC-30	LLM providers	Retired model name reported with the provider's message	Provider returning HTTP 400	Outcome contains the decommissioned-model message	Pass
TC-31	LLM providers	Diagnostics reach the explainability report	Rejected key during a real generation	Report falls back to template and records the failure	Pass
TC-32	Multimodal	PDF tables become structured text	Generated PDF with a table	Row text contains the header-value pairs	Pass
TC-33	Multimodal	Figures classified and described	Generated PDF with a chart image	Classified as chart or diagram; no fabricated caption	Pass
TC-34	API	Full round trip over HTTP	Multi-subject upload through the test client	Job completes; topic, subtopics, explainability and completion all succeed	Pass
TC-35	API	Completion persisted and returned in the job snapshot	Completion posted with accuracy 88	Stored and reported as completed with accuracy 88	Pass
TC-36	API	Out-of-range accuracy clamped	Completion posted with accuracy 250	Stored as 100	Pass
TC-37	Front end	Lesson advances from theory into the quiz	Rendered lesson, Continue activated	Quiz stage active with options rendered	Pass
TC-38	Front end	Complete activity sequence scores correctly	All four activities answered correctly	Accuracy 100 per cent at the score stage	Pass
TC-39	Front end	Completion posts to the backend and updates local state	Topic marked complete	Completion request issued; store updated	Pass
TC-40	Front end	Trust score reaches the lesson view	Explainability stubbed with a score	Trust percentage displayed on the lesson	Pass

8.4 Test Execution Results

The complete backend suite comprises 65 automated tests distributed across seven files. The final execution reported 65 passed and no failures or errors, in approximately five minutes and forty-three seconds. The front-end harness reports 13 of 13 checks passing.

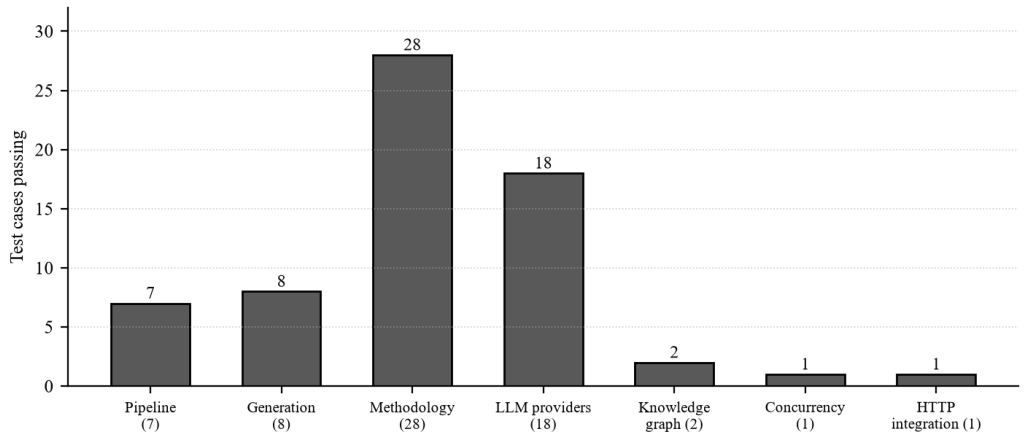


Fig 8.1: Distribution of the 65 backend tests by test file

Table 8.3: Backend test distribution and outcome

Test file	Tests	Principal coverage
test_pipeline.py	7	Parsing, sequencing, prerequisites, collision handling, failure recording
test_generation.py	8	Subtopic rows, caching, regeneration, image presence and absence
test_methodology_pipeline.py	28	Retrieval, Shapley mathematics, validation metrics, optimisation, multimodal extraction
test_llm_providers.py	18	Provider request and response handling, failure diagnostics, trust comparison on a real upload
test_knowledge_graph.py	2	Graph and roadmap endpoints
test_concurrency.py	1	Duplicate generation suppression under parallel access
test_http_integration.py	1	End-to-end round trip over the HTTP interface
Total	65	All passing

8.5 Defects Identified and Corrected

Testing identified a number of defects, several of which would have materially affected the system's output. They are recorded here because the corrections form part of the engineering result.

Table 8.4: Defects identified during testing and their correction

Defect	Effect	Correction
Undeclared shuffle seed in the lesson screen	A reference error was raised the moment the learner left the theory stage, so all four activity types failed to render. The code was syntactically valid, so static checking did not detect it.	Seed declared per round and re-rolled on regeneration; a Node-based harness was added that executes the real modules and reproduces the fault if reintroduced.
Default cross-origin policy omitted the documented port	Every browser request from the front end was blocked before reaching a route, on a stock installation.	The documented port added to the default allowed origins, for both host spellings.
Shared buffer across extraction strategies	A partially successful first extraction left its text in place for the next strategy to append to, duplicating every page in the extracted document and therefore in the retrieval evidence.	Each strategy writes to its own buffer.
Test fixtures written into the application's data directories	Fixture chunks accumulated in the live retrieval index and fixture files in the live upload directory, contaminating the evidence used for real lessons.	Index and upload directories made configurable and redirected during testing; isolation verified after each run.
Non-deterministic image	A test asserted that no image was found, which held only on a machine without	Lookup stubbed deterministically, with a second test covering the case where an

Defect	Effect	Correction
test	internet access and failed on any machine with it.	image is found.
Cascading fixture failure on Windows	A test that failed part-way left its database session open, so the next fixture could not delete the file and every subsequent test reported an error, obscuring the original failure.	Teardown disposes the pool, collects leaked sessions, retries, and falls back to dropping the tables.
Silent provider failures	A rejected API key, a retired model identifier and an unconfigured system all produced an identical result, so a misconfiguration was indistinguishable from intended behaviour.	Per-provider outcomes recorded with the provider's own message and surfaced in the report.
Unlatched embedding-model failure	Every call retried an expensive model load, so an unreachable model host made each lesson take minutes instead of degrading immediately.	Load failure cached and raised as a typed exception on subsequent calls.
Completion data discarded	The completion endpoint acknowledged accuracy but stored nothing, so progress existed only in browser storage.	Completion timestamp and clamped accuracy persisted and returned in the job snapshot.

8.6 Validation of the Evaluation Layer

Because the evaluation layer is the project's principal contribution, it was validated by controlled comparison in addition to unit testing. The same document was uploaded and processed through the real pipeline, and the language-model layer was then driven to return two different lessons for the same topic: one restating the uploaded material, and one describing an unrelated subject and containing a fabricated numeric value. Every other variable - the uploaded file, the retrieved evidence, the provider path and the scoring code - was held constant. The results are reported in Section 9.3.

CHAPTER IX
RESULTS AND DISCUSSION

9.1 Implemented Functionality

The following functionality is implemented and verified. The status column distinguishes what has been confirmed by execution from what is implemented but could not be verified in the development environment.

Table 9.1: Implemented functionality and its verification status

Capability	Status	Evidence
Upload and text extraction (PDF, text, image)	Working	Round-trip test; multimodal extraction tests on a generated PDF
Curriculum parsing, single and multi-subject	Working	Parsing tests; live upload produced two subjects with four units
Topic identifier generation and collision resolution	Working	Duplicate-upload test produces disjoint identifier sets
Knowledge graph and roadmap derivation	Working	Ordering and prerequisite tests; roadmap endpoint test
Chunking, embedding and per-subject indexing	Working	Live run reported hybrid retrieval with vector search active
Hybrid retrieval with fusion and re-ranking	Working	Retrieval tests in both hybrid and keyword-only modes
Lesson generation via provider chain	Working with a stubbed provider	18 provider tests cover request construction, parsing and failure modes
Template fallback with verbatim evidence	Working	Template tests with and without evidence
Explainability: attribution, relevance, Shapley, trace	Working	Axiomatic and brute-force verification; live report populated
Validation metrics and trust score	Working	Controlled comparison in Section 9.3
Activity generation and scoring	Working	Front-end harness completes all four activities at full accuracy
Completion persistence	Working	Verified live and in the HTTP round-trip test
Live call to a commercial LLM provider	Not verified	Requires a provider key and outbound access; every layer below the network call is tested
Neural image captioning for figures	Not verified	Requires model weights that could not be obtained; figures receive measured descriptions
Optical character recognition for scanned PDFs	Not verified	Optional package not installed; the failure path returns a readable message

9.2 System Operation

A live execution was performed with the backend and front end running as separate local processes. A two-subject syllabus was uploaded through the HTTP interface and progressed through the extract, distribute and generate phases to completion. The resulting curriculum contained two subjects of two units each. A topic request returned a lesson with four sections and the full set of activities, and the explainability endpoint returned a populated report.

The retrieval trace from that run reported the method as hybrid with both vector and keyword retrieval active, two chunks selected, and attribution assigned to the uploaded file by name. Cross-origin pre-flight from the documented front-end origin returned the expected permissive headers, confirming the configuration correction described in Section 8.5.

Screenshots of the running interface are to be inserted by the candidate at submission; they are not reproduced here because the figures in this report are limited to material that could be generated and verified directly. The recommended set is the upload screen, the extraction progress screen, the subject and unit views, a rendered lesson showing the trust chip, and the expanded explainability report.

9.3 Evaluation of the Trust Scoring Layer

The controlled comparison described in Section 8.6 produced the results in Table 9.2 and Figure 9.1. Both lessons were generated for the same topic, against the same uploaded document and the same retrieved evidence; only the generated content differed.

Table 9.2: Trust metrics for a grounded and an off-topic lesson on identical evidence

Metric	Lesson A - restates the uploaded material	Lesson B - off-topic content
Trust score	0.889	0.052
Trust band	High	Low
Groundedness	0.643	0.025
Faithfulness	1.000	0.000
Hallucination risk	0.000	1.000
Semantic similarity	0.699	0.000
Topic alignment	0.610	0.000
Sentence verdicts	3 supported, 0 weak, 0 unsupported	0 supported, 0 weak, 3 unsupported
Similarity method	Embedding	Embedding
Evidence source	The uploaded document, by filename	The uploaded document, by filename

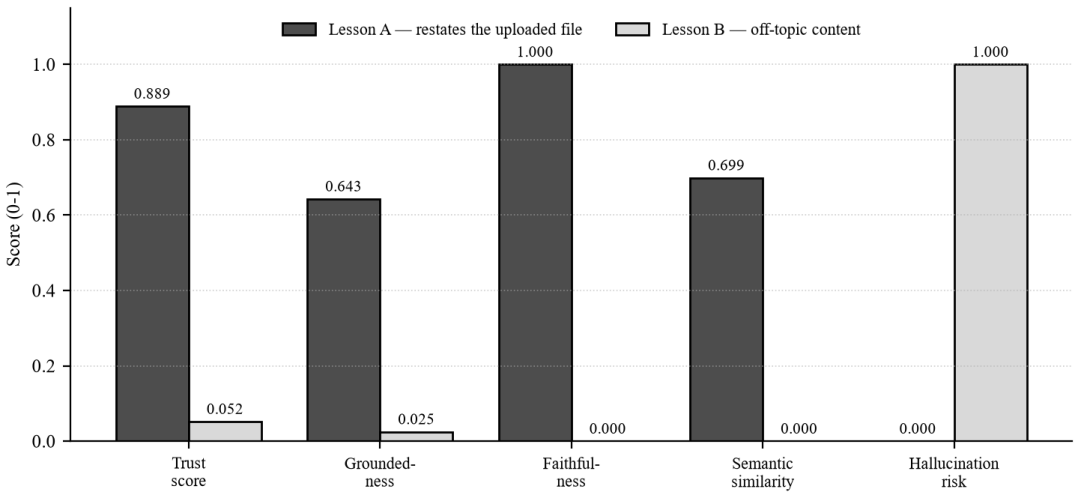


Fig 9.1: Measured trust metrics for a grounded against an off-topic lesson

9.3.1 Interpretation

The separation between the two lessons is unambiguous across every metric, and in the intended direction. The grounded lesson attains a trust score of 0.889 and the high band, with faithfulness of 1.000, meaning every one of its sentences cleared the support threshold against the retrieved evidence. The off-topic lesson attains 0.052 and the low band, with faithfulness of 0.000 and hallucination risk of 1.000.

Two specific behaviours are worth drawing out. First, the sentence-level trace for the

grounded lesson linked each sentence to the uploaded document by filename, with individual similarities of 0.79, 0.63 and 0.51 - that is, the system reports not merely that the lesson is grounded but which sentence rests on which passage, and how firmly. Second, the numeric check operated as designed: the fabricated value in the off-topic lesson was flagged with the reason "number(s) not found in evidence: 4096", identifying the specific value rather than merely marking the sentence unsupported.

The Shapley analysis for the grounded lesson distributed the groundedness across the two retrieved chunks in shares of 0.844 and 0.156, identifying the passage the lesson actually depended on. Because the values sum to the total groundedness of the full evidence set, this distribution is exact rather than approximate.

The low-trust path was also exercised: the off-topic lesson triggered the single strict regeneration described in Section 5.4.4, and because the regenerated draft did not score higher, the first draft was retained and the decision recorded in the report.

9.3.2 Limits of the Measurement

These results demonstrate that the layer separates grounded from ungrounded content on this comparison. They should not be read as an accuracy figure for the system as a whole. The comparison uses two deliberately constructed lessons at opposite extremes; it establishes that the metric responds correctly and strongly to the condition it measures, not how the metric behaves across the continuum of partially grounded content, which would require a labelled corpus and a larger study. No such study was performed, and no accuracy claim beyond this comparison is made.

9.4 Behaviour Without Source Prose

A separate result concerns the system's behaviour when the uploaded material contains no prose to ground against. A syllabus consisting only of unit headings and topic names was uploaded. Retrieval succeeded and reported the hybrid method, and the lesson was produced from the template. The validation layer then reported the trust score as unavailable, with the stated reason that the template makes no claims about the subject and there is therefore nothing to validate.

This is the designed behaviour and it is worth stating explicitly, because it is the outcome most likely to be mistaken for a defect. The alternative - reporting zero - would assert that the lesson was assessed and found entirely unsupported, which is a different and false claim. The distinction between "not measurable" and "measured as poor" is preserved throughout the metric set.

9.5 Verification Results

The automated suite reports 65 of 65 backend tests passing and 13 of 13 front-end checks passing. Isolation was confirmed after the final run: the application's database file, retrieval index directory and upload directory were all absent or empty, demonstrating that the suite wrote exclusively to its own locations.

The front-end harness was additionally validated negatively. With the corrected declaration removed, the harness fails at the exact line and reports the reference error, confirming that it detects the class of defect it was written for rather than passing vacuously.

9.6 Comparison Against the Stated Objectives

Table 9.3: Achievement of the stated objectives

Objective (Section 1.5.2)	Status	Outcome
1. Document processing	Achieved	Staged extraction across three formats; tables and figures recovered as retrievable text
2. Curriculum parsing	Achieved	Single and multi-subject documents parsed; identifiers unique and collision-resolved
3. Prerequisite sequencing	Achieved	Graph constructed and topologically ordered; sequence and prerequisite stored per topic
4. Hybrid retrieval	Achieved	Vector and BM25 retrieval fused, re-ranked and selected under budget; verified in both modes
5. Grounded generation	Achieved with one qualification	Prompt construction, provider chain and template fallback all implemented and tested; the network call to a commercial provider remains unverified
6. Explainable AI analysis	Achieved	Attribution, relevance, exact Shapley values and sentence-level tracing, verified against the formal definition
7. Responsible AI validation	Achieved	Full metric set and composite trust score; demonstrated to separate grounded from ungrounded content by 0.889 against 0.052
8. Graceful degradation	Achieved	Fallbacks for embeddings, OCR, providers and images, each exercised by tests and reported in the output
9. Verification	Achieved	65 backend tests and 13 front-end checks, all passing, running offline and deterministically

9.7 Known Issues and Limitations of the Result

- **Provider network path unverified.** Every layer beneath the network call is covered by tests, including request construction, response parsing and each failure mode, but no call to a commercial provider endpoint was completed from the development environment.
- **Single-process concurrency.** Duplicate-generation suppression relies on an in-process lock and would not hold across multiple server processes.
- **Parsing depends on recognisable structure.** Syllabi with unconventional formatting may yield a degraded unit or topic breakdown; the parser falls back progressively but cannot recover structure that is not present.
- **Heuristic components remain heuristic.** Image-type classification and duration estimation are rule-based and are labelled as such wherever displayed.
- **Evaluation scope.** The trust layer was validated by controlled comparison, not against a labelled corpus; no general accuracy figure is claimed.
- **Synchronous pre-generation.** With pre-generation of every topic enabled, upload completion waits for one generation call per topic, which is slow on a large syllabus. The default pre-generates only each unit's first topic.

CHAPTER X
CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

This project set out to address a specific gap: learners possess a syllabus that specifies their curriculum but supplies no material, and the AI tools they increasingly use to fill that gap provide no way to tell whether what they produce is supported by the learner's own prescribed documents. The work has delivered a complete, working system that closes that gap end to end.

The implemented platform converts an uploaded syllabus into a structured, prerequisite-ordered learning path without manual data entry, indexes the learner's material for retrieval, and generates lessons through a pipeline that retrieves evidence by both dense and lexical means before any text is generated. Each lesson is then analysed: the contribution of each piece of evidence is quantified by exact Shapley values, each generated sentence is traced to the passage that best supports it, and the whole is scored against groundedness, faithfulness, hallucination risk and semantic similarity to produce a single trust value with a declared band.

The central claim of the project is supported by measurement. On a controlled comparison holding the uploaded document, the retrieved evidence and the scoring code constant, a lesson restating the source material scored 0.889 in the high band while an off-topic lesson scored 0.052 in the low band, with faithfulness of 1.000 against 0.000, and a fabricated numeric value was identified by value. Generated study material can therefore be bound to a learner's own documents, and the strength of that binding can be measured and displayed rather than assumed.

A secondary outcome concerns engineering discipline rather than novelty. The system was built to report its own degradation: where the embedding model is unavailable it says that retrieval ran lexically; where a provider is misconfigured it reports the provider's own error; and where no evidence exists it reports that no score can be computed instead of returning zero. Several of the defects recorded in Section 8.5 were failures of exactly this kind - silent fallbacks that made a broken configuration indistinguishable from normal operation - and correcting them was as substantial a part of the work as implementing the metrics themselves.

10.2 Objectives Achieved

All nine specific objectives stated in Section 1.5.2 were achieved, with one qualification recorded against objective 5: the network call to a commercial language-model provider could not be exercised from the development environment, although every layer beneath that call is covered by automated tests. The detailed assessment is given in Table 9.3.

10.3 Technical Contributions

1. A complete syllabus-to-lesson pipeline requiring no manual curriculum entry, from document upload through to an activity-bearing lesson.
2. A native exact Shapley-value implementation over retrieved evidence, verified against the brute-force definition and the efficiency, symmetry and dummy-player axioms, with permutation sampling beyond a configured bound.
3. A composite trust score with published weights, declared bands and explicit

unavailability where a metric cannot be computed.

4. A method-aware similarity backend with thresholds calibrated separately for embedding and lexical modes, reporting which mode was used.
5. A verified degradation strategy in which every optional dependency has a tested fallback that is reported in the user-visible output.
6. A dependency-free front-end test harness capable of executing the real interface modules outside a browser.

10.4 Future Scope

10.4.1 Immediate Extensions

- **Verification against commercial providers.** Completing a live call with a provider key would close the one remaining gap in the verification of objective 5.
- **Cross-encoder re-ranking.** Replacing the lightweight re-ranker with a cross-encoder would improve ordering precision at a measurable computational cost, and the existing trace makes the comparison straightforward.
- **Evaluation against a labelled corpus.** Measuring the trust layer against human-annotated groundedness labels would convert the present demonstration into a quantified accuracy claim.

10.4.2 Scalability and Deployment

- **Multi-process deployment.** Moving the per-topic generation lock to a shared store would permit the system to run behind a load balancer; the current lock is correct only within one process.
- **Database migration.** The persistence layer already branches its connection configuration by engine, so moving to PostgreSQL for institutional use is a configuration change plus a migration.
- **Asynchronous generation.** Moving generation to a task queue would remove the synchronous wait that pre-generation of every topic currently imposes on upload completion.

10.4.3 Functional Extensions

- **Multi-user accounts and cohort analytics.** The user table exists but is unused. Populating it would support per-student progress and allow an instructor to see which topics are most often flagged as weakly supported.
- **Curriculum-wide quality auditing.** Because every lesson stores its own report, the stored reports can be aggregated to identify syllabus areas where the available material is too thin to support reliable generation - information useful to a department, not only to a learner.
- **Adaptive sequencing.** Completion accuracy is already recorded per topic; feeding it back into the knowledge graph would allow the path to be reordered according to demonstrated mastery.
- **Richer multimodal retrieval.** Table and figure extraction is implemented; adding captioning and diagram-aware retrieval would allow figures to be retrieved as first-class evidence.
- **Assessment export.** The activity generator could produce printable question papers carrying the evidence citation for each question.

10.5 Concluding Remark

The value of a generated explanation to a student depends less on its fluency than on whether it can be checked. This project has shown that the checking can be automated, quantified and placed in front of the learner at the point of use, and that doing so costs little more than the retrieval the system already performs. A study tool that tells a learner which of its own sentences to verify is more useful, and considerably safer, than one that presents every sentence with equal confidence.

REFERENCES

[1] M. T. Ribeiro, S. Singh and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 1135-1144.

[2] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, 2017, pp. 4765-4774.

[3] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020, pp. 9459-9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[4] L. S. Shapley, "A value for n-person games," in Contributions to the Theory of Games, vol. II, H. W. Kuhn and A. W. Tucker, Eds. Princeton, NJ, USA: Princeton Univ. Press, 1953, pp. 307-317.

[5] S. E. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333-389, 2009.

[6] G. V. Cormack, C. L. A. Clarke and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval, Boston, MA, USA, 2009, pp. 758-759.

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. 2019 Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), Hong Kong, 2019, pp. 3982-3992. [Online]. Available: https://arxiv.org/abs/1908.10084

[8] J. Johnson, M. Douze and H. Jegou, "Billion-scale similarity search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535-547, 2021. [Online]. Available: https://arxiv.org/abs/1702.08734

[9] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1-38, 2023. [Online]. Available: https://arxiv.org/abs/2202.03629

[10] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in Proc. 8th Int. Conf. Learning Representations (ICLR), 2020. [Online]. Available: https://arxiv.org/abs/1904.09675

[11] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, 2017, pp. 5998-6008.

[12] A. A. Hagberg, D. A. Schult and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in Proc. 7th Python in Science Conf. (SciPy2008), Pasadena, CA, USA, 2008, pp. 11-15.

[13] S. Ramirez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/ (accessed Oct. 2025).

[16] Explosion AI, "spaCy: Industrial-strength natural language processing." [Online]. Available: https://spacy.io/ (accessed Oct. 2025).

[17] Artifex Software, "PyMuPDF documentation." [Online]. Available: https://pymupdf.readthedocs.io/ (accessed Oct. 2025).

APPENDIX A: SCREENSHOT OF THE PROJECT



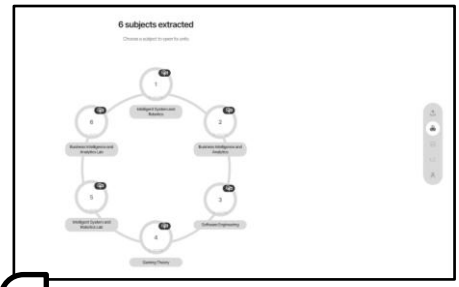
1 - Syllabus Uploading Page



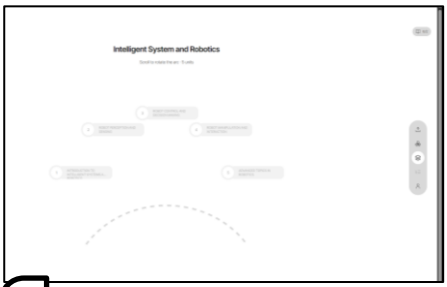
2 – Uploading file (Progress bar)



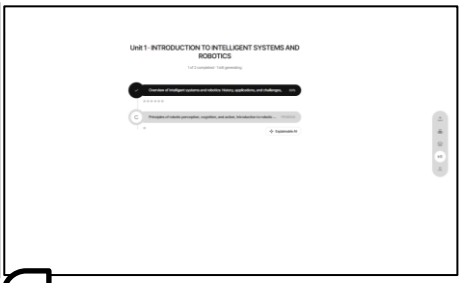
11 – Content Exaction & Distribution



10 – Structured Multi-Subject View



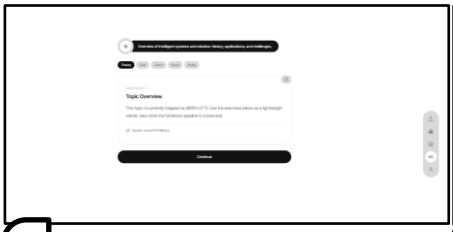
9 – Multi-Unit View for Subject-wise



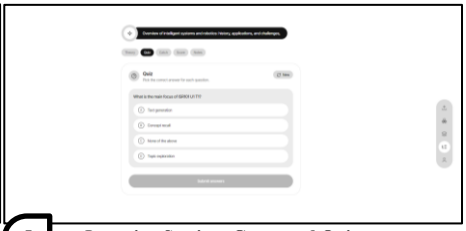
8 – Topic View of Unit



7 – Explainability: User have to upload source document to get the Trust Score



6 – Learning Session: Topic Definition, Overview .



5 – Learning Session: Generated Quiz.



3 – Score: For as per the quiz



4 – Document of Full Answer with terminology of the Topic

APPENDIX B: CONFIGURATION REFERENCE

Table 10.1: Configuration settings

Setting	Default	Purpose
DATABASE_URL	sqlite:///./sya.db	Database connection; a PostgreSQL URL may be substituted
UPLOAD_DIR	./uploads	Directory holding uploaded documents
FAISS_INDEX_DIR	./faiss_indexes	Per-subject retrieval indexes
CORS_ORIGINS	Local origins on ports 5500, 5173, 3000, 8080	Browser origins permitted to call the API; must include the origin actually used
LLM_PROVIDER_ORDER	ollama, mistral, groq, gemini	Order in which providers are attempted; the first valid response is used
GROQ_API_KEY / GROQ_MODEL	unset / llama-3.1-8b-instant	Groq credentials and model
MISTRAL_API_KEY / MISTRAL_MODEL	unset / mistral-small-latest	Mistral credentials and model
GEMINI_API_KEY / GEMINI_MODEL	unset / gemini-1.5-flash	Gemini credentials and model
OLLAMA_BASE_URL / OLLAMA_MODEL	http://localhost:11434 / llama3.2	Local provider endpoint and model
PREGENERATE_UNIT_ENTRY_TOPICS	true	Pre-generates the first topic of each unit after upload
PREGENERATE_NEXT_ON_COMPLETE	true	Pre-generates the following topic when one is completed
PREGENERATE_ALL_TOPICS	false	Generates every topic at upload time; one provider call per topic

Hosted model identifiers are retired by providers from time to time. Where a model name is no longer valid the system records the provider's own error message and reports it in the explainability output, so that a stale configuration is distinguishable from an unconfigured system.